

Final Year Design Project

PoseFit: A Web-Based Fitness Assistant for Home Users



By

Maham Mobeen 2022-VGR-137

Shanza Kahsaf 2022-VGR-125

Mahnoor 2022-VGR-142

**Under the supervision of
Prof. Mrs. Nafeesa Rashid**

Bachelor of Science in Information Technology

Session (2022-2026)

Department of Information Technology,

Govt. Vigar-un-Nisa,

Graduate College for Women.

PoseFit: A Web-Based Fitness Assistant for Home Users

**A project presented to
University of the Punjab, Lahore.**

**In partial fulfilment of the
requirement for the degree of**

**Bachelors of Science in Information Technology
Session (2022-2026)**

By

Maham Mobeen	2022-VGR-137
Shanza Kashaf	2022-VGR-125
Mahnoor	2022-VGR-142

**Department of Information Technology,
Govt. Viqar-un-Nisa,
Graduate College for Women.**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Signature: -----

Maham Mobeen [088147]

Signature: -----

Mahnoor [088150]

Signature: -----

Shanza Kashaf [088152]

CERTIFICATE OF APPROVAL

It is to certify that the final year project (FYP) of BSIT, session (2021-25), the project **PoseFit: A Web-Based Fitness Assistant for Home Users** was developed by **Maham Mobeen (2022-VGR-137), Shanza Kashaf (2022-VGR-125), Mahnoor (2022-VGR-142)** under the supervision of **Prof. Mrs. Nafeesa Rashid** in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in **Information Technology**.

Signature: -----

FYDP Supervisor: Prof. Mrs. Nafeesa Rashid

Signatures (Faculty Advisory Committee (FAC))

Signatures			
Name	Prof. Mrs. Nafeesa Rashid		
	FAC1	FAC2	FAC3

Signature: -----

Head of FYDP Coordination Office: Prof. Mrs. Nafeesa Rashid.

Signature:-----

Dated: _____

Chairperson, Department of Information Technology

Executive Summary

PoseFit: A Web-Based Fitness Assistant for Home Users is a web application designed to help people maintain their health and fitness from the comfort of home. The system provides three main features: a chatbot that answers common fitness questions, an AI-based diet plan generator that suggests meals based on body measurements and goals, and a posture detection tool that uses a webcam to ensure exercises are performed correctly.

PoseFit is designed to be simple, accessible, and easy to use, especially for individuals who do not have access to gyms or personal trainers. It focuses on basic bodyweight exercises such as squats, plank, arm raises, and side bends. For diet planning, the system includes an AI-based diet plan generator that uses health metrics to generate personalized plans for three fixed goals: weight loss, muscle gain, and maintenance. The chatbot supports quick and interactive responses, while the application works smoothly on both computers and mobile devices.

The system is built using modern web technologies including MongoDB, Express, React, and Node, along with supporting libraries for chatbot, diet, and posture functionalities. It is designed to be fast, reliable, and easy to maintain or upgrade.

Unlike large commercial fitness applications, PoseFit keeps things practical and user-friendly. It avoids complicated workout programs or unnecessary features, instead focusing on safe exercises and easy-to-follow diet plans tailored for home users.

In summary, PoseFit is a lightweight yet effective fitness assistant that helps users stay active, eat healthier, and exercise with proper posture, all in a convenient, home-based environment.

Keywords:

1. Web-based Fitness Assistant
2. Posture Detection
3. AI- based Diet Plan Generator

Acknowledgement

We express our sincere gratitude to Allah Almighty for enabling us to complete this Final Year Project successfully. We are thankful to our project supervisor for their super guidance, support and valuable feedback throughout the project. Lastly, we thank our families and friends for their continuous encouragement and support during this journey.

Signature: -----

Maham Mobeen [088147]

Signature: -----

Mahnoor [088150]

Signature: -----

Shanza Kashaf [088152]

Abbreviations

Table of all the abbreviations and acronyms used in your FYP along with their respective brief description. The abbreviation list must be ordered alphabetically.

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
BMI	Body Mass Index
BMR	Basal Metabolic Rate
ERD	Entity Relationship Diagram
IT	Information Technology
JWT	JSON Web Token
MERN	MongoDB, Express, React, Node
NLP	Natural Language Processing
NoSQL	Non-Relational Database
SRS	Software Requirements Specification
SDS	Software Design Specification
TDEE	Total Daily Expenditure
UI	User Interface
UX	User Experience
UML	Unified Modeling Language

Contents

Chapter 1 Introduction.....	14
1. Introduction.....	15
1.1 Problem Statement.....	15
1.2 Problem Solution	15
1.3 Objectives of the Proposed System	16
1.4 Scope.....	16
1.5 System Components.....	18
1.5.1 Authentication Module (Client-Side Module).....	18
1.5.2 User Dashboard Module (Client-Side Module)	18
1.5.3 Chatbot Module (Client-Side Module).....	18
1.5.4 Health Metrics Module (Client-Side Module)	18
1.5.5 Diet Plan Module (Client-Side Module)	19
1.5.6 Posture Detection Module (Client-Side Module)	19
1.5.7 Admin Module	19
1.6 Related System Analysis/Literature Review	20
1.7 Vision Statement.....	20
1.8 System Limitations and Constraints	21
1.9 Tools and Technologies	22
1.10 Project Deliverables	23
1.11 Project Planning.....	23
1.12 Summary	24
Requirements Analysis.....	25
2. Analysis	26
2.1 User classes and characteristics	26
2.2 Requirement Identifying Technique	26
2.3 Functional Requirements.....	27
2.3.1 Functional Requirement 1.....	27
2.3.2 Functional Requirement 2.....	28
2.3.3 Functional Requirement 3.....	28
2.3.4 Functional Requirement 4.....	29
2.3.5 Functional Requirement 5.....	30

2.3.6 Functional Requirement 6.....	30
2.3.7 Functional Requirement 7.....	31
2.3.8 Functional Requirement 8.....	32
2.3.9 Functional Requirement 9.....	33
2.3.10 Functional Requirement 10.....	34
2.3.11 Functional Requirement 11.....	34
2.3.12 Functional Requirement 12.....	35
2.3.13 Functional Requirement 13.....	36
2.3.14 Functional Requirement 14.....	36
2.4 Non-Functional Requirements.....	37
2.4.1 Reliability	37
2.4.2 Usability	37
2.4.3 Performance	37
2.4.4 Security	38
2.4.5 Portability	38
2.4.6 Maintainability	38
2.4.7 Scalability	38
2.5 External Interface Requirement.....	38
2.5.1 User Interface Requirements	39
2.5.2 Software Interfaces	40
2.5.3 Hardware Interfaces	40
2.5.4 Communication Interfaces.....	40
2.6 Use Case Analysis	41
2.6.1 Use Case 1.....	41
2.6.2 Use Case 2.....	42
2.6.3 Use Case 3.....	43
2.6.4 Use Case 4.....	45
2.6.5 Use Case 5.....	46
2.6.6 Use Case 6.....	47
2.6.7 Use Case 7.....	48
2.6.8 Use Case 8.....	49
2.6.9 Use Case 9.....	50
2.6.10 Use Case 10.....	52

2.6.11 Use Case 11.....	53
2.6.12 Use Case 12.....	54
2.7 Use Case Diagram	55
2.8 Summary	56
Chapter 3 Design and Architecture	57
3. System Design.....	58
3.1 Design considerations	58
3.1.1 Assumptions and Dependencies	58
3.1.2 Limitations.....	59
3.1.3 Risks.....	59
3.2 Design Models.....	59
3.2.1 Class Diagram	60
3.2.2 Sequence Diagram	61
3.2.2 State Transition Diagram	68
3.3 Architectural Design	71
3.3.1 UML Class Relationship Diagram.....	72
3.3.2 UML Component Diagram	72
3.3.3 Layered Architecture (3-Tier).....	73
3.3.4 MVC	73
3.4 Data Design	74
3.4.1 ER Diagram.....	74
3.4.2 MongoDB Data Structure.....	75
3.4.3 Data Processing and Organization.....	75
3.4.4 Data Dictionary	75
1- Object: User	78
2- Object: HealthMetrics	79
3- Object: DietPlan.....	80
4- Object: ChatMessage	80
5- Object: Feedback	81
3.5 User Interface Design	81
3.5.1 Screen Images	82
3.5.2 Screen Objects and Actions	84
3.6 Behavioral Model	86

3.7 Design Decisions	87
3.8 Summary	89

TABLES

Table 1: Related System Analysis with proposed project solution	20
Table 2: Tools and Technologies for proposed system	22
Table 3: User class and characterization.....	26
Table 4: FR-1	27
Table 5: FR-2	28
Table 6: FR-3	28
Table 7: FR-4.....	29
Table 8: FR-5	30
Table 9: FR-6	30
Table 10: FR-7	31
Table 11: FR-8.....	32
Table 12: FR-9	33
Table 13: FR-10	34
Table 14: FR-11	34
Table 15: FR-12	35
Table 16: FR-13	36
Table 17: FR-14	36
Table 18: UC-1.....	41
Table 19: UC-2.....	42
Table 20: UC-3.....	43
Table 21: UC-4.....	45
Table 22: UC-5.....	46
Table 23: UC-6.....	47
Table 24: UC-7.....	48
Table 25: UC-8.....	49

Table 26: UC-9.....	50
Table 27: UC-10.....	52
Table 28: UC-11.....	53
Table 29: UC-12.....	54
Table 30: Risks and Mitigation Strategy	59
Table 31: Layered Architecture with Responsibilities	73
Table 32: MVC Components	73
Table 33: Data Dictionary.....	75

FIGURES

Figure 1: Gantt Chart.....	24
Figure 2: Use Case Diagram.....	55
Figure 3: Class Diagram.....	60
Figure 4: Sequence Diagram 1.....	61
Figure 5: Sequence Diagram 2.....	62
Figure 6: Sequence Diagram 3.....	63
Figure 7: Sequence Diagram 4.....	64
Figure 8: Sequence Diagram 5.....	65
Figure 9: Sequence Diagram 6.....	66
Figure 10: Sequence Diagram 7.....	67
Figure 11: State Transition Diagram 1.....	68
Figure 12: State Transition Diagram 2.....	69
Figure 13: State Transition Diagram 3.....	70
Figure 14: State Transition Diagram 4.....	71
Figure 15: UML Class Relationship Diagram	72
Figure 16: UML Component Diagram.....	72
Figure 17: ER Diagram.....	74
Figure 18: Landing, Register, and Login Page	82
Figure 19: Dashboard and Diet Plan Page.....	83
Figure 20: Posture Page.....	84

Figure 21: Sequence Model	86
Figure 22: State Transition Model	87

Chapter 1 Introduction

1. Introduction

The increasing adoption of home-based workout routines without professional trainers has introduced several challenges for individuals, including maintaining correct exercises posture, following a healthy diet, and finding trustworthy fitness guidance. To address these challenges, PoseFit a web application has been developed with three key tools, a fitness chatbot for quick response to user queries, an AI-based diet plan generator trained to provide personalized plans based on the user's BMI and goals (weight loss, muscle gain, or maintenance) and a posture detector that uses the user's webcam to provide guidance through 3-4 basic exercises. This application is built using modern web technologies to ensure usability and efficiency. The scope and limitations of the system are also discussed, along with the potential improvements for future development.

1.1 Problem Statement

In today's busy world many people struggle to maintain their health due to the lack of time, money, or proper guidance. Those who exercise at home often use incorrect posture which leads to pain or severe injuries. Similarly, many basic diet plans that ignore personal preferences are hard to follow consistently. To address these issues, PoseFit is proposed as a web application that offers a fitness chatbot for basic guidance, an AI-based diet plan generator based on common goals (weight loss, muscle gain, or maintenance), and posture detection for simple exercises using a webcam. PoseFit aims to make safe, affordable, and personalized fitness support accessible from home.

1.2 Problem Solution

To address these challenges, PoseFit was developed as a web-based fitness assistant for home users. The purpose of the app is to help people stay fit without spending too much money. PoseFit offers three core features a fitness chatbot that provides simple and clear response to users' queries and saves time by eliminating the need to search online for every question. An AI-based diet plan generator that generates simple diet plans for three goals (weight loss, muscle gain, and maintenance). A posture detection tool that uses the webcam to monitor form during basic exercises. By combining fitness guidance, personalized diet plans, and posture feedback in one free and easy to use platform, PoseFit helps users exercise safely and eat better without needing a gym

or personal trainer.

1.3 Objectives of the Proposed System

Chatbot Accuracy:

The system includes a fitness chatbot that answers common questions. It aims to correctly respond to at least 80% of the basic fitness queries by the end of the project

Diet Plan Generator:

PoseFit provide a diet plan generator with three fixed goals (weight loss, muscle gain and maintenance). Diet plans will be generated and displayed in under 2 second after input.

Posture Detection Module:

Using the webcam the system will monitor the form during 3-4 basic exercises like squats, planks, arm raises and side bends. It will offer feedback on posture with at least 75% accuracy.

User privacy and security:

Personal data such as age, weight, height and BMI will be kept private and no user data will be shared externally or secured insecurely.

Platform Accessibility:

PoseFit will be developed as a MERN based web application. It will be a responsive web application that works smoothly on both laptops and smartphones by final deployment.

1.4 Scope

The scope of PoseFit tells us which features are part of the system and which are not. PoseFit is a fitness website for people between 16 to 50 years, who want to exercise at home easily without depending on personal trainers.

In-Scope (Features to Deliver):

- A chatbot that answer basic fitness questions with about 80% accuracy.
- A BMI based diet plan for people who want to lose weight, gain muscle or stay fit.
- A webcam-based posture detection feature that detect body posture for simple exercise like planks, arm raise, squat and side bend.

- A responsive web app built with the MERN stack that works on both desktop and mobile devices.

Out-of-Scope (Not Included):

- The app does not include any special or customized workout.
- No medical or clinical nutrition advice is provided.
- The system does not detect posture for difficult or advanced exercises.
- Exercise videos cannot be recorded or saved.

Stakeholders:

- **Main users:** people between 16 to 50 years, who want easy and low-cost way to stay fit at home.
- **Project team:** students and developers building the system.
- **Supervisor:** guides the team and checks the quality of the work.

Constraints:

- Posture accuracy depends on webcam quality and lighting.
- Due to limited time and resources only basic features are included.
- All modules need an internet connection to work properly.

Milestones:

- Planning: Sep 1 – Sep 19, 2025
- Requirement Gathering: Sep 20 – Oct 15, 2025
- Analysis: Oct 16 – Nov 10, 2025
- Designing: Nov 11, 2025 – Jan 15, 2026
- Coding: Jan 16 - april 30 , 2026
- Testing: May 1 – May 31, 2026
- Maintenance: Jun 1 - Jun 30, 2026

1.5 System Components

The PoseFit is web application that consists of the following components:

1.5.1 Authentication Module (Client-Side Module)

This module handles user registration and login so users can safely access the system.

Features:

- Login with email and password.
- Signup with email verification
- Google login
- Forget password option

1.5.2 User Dashboard Module (Client-Side Module)

This module provides users a central space for users to access all features.

Features:

- Provides access to Chatbot, Health Metrics, Diet plan, and Posture detection.
- Update personal information via setting page.
- Submit feedback about the system.
- Displays saved chat history, diet plans, and posture usage.

1.5.3 Chatbot Module (Client-Side Module)

This module allows users to talk with the fitness bot and get basic guidance.

Features:

- Simple chat interface.
- Send and receives fitness questions and answers.
- Saves limited user chat history in the database.

1.5.4 Health Metrics Module (Client-Side Module)

This module calculates user fitness metrics separately from diet plans.

Features:

- Take inputs (height, weight, age, gender, activity level, and goal).
- Calculate BMI, BMR, TDEE, and Macros.
- Maintains only one saved health metrics record per user for diet plan generation.

1.5.5 Diet Plan Module (Client-Side Module)

This module generates diet plans based on previously calculated BMI, BMR, TDEE, user preference and the selected fitness goals.

Features:

- Collects user preferences through follow-up questions.
- Diet plan generation depends on calculated metrics.
- Save generated diet plans.

1.5.6 Posture Detection Module (Client-Side Module)

This module helps users perform exercises correctly with posture monitoring.

Features:

- Use webcam to detect body position.
- Tracks pose using mediaPipe.
- Give quick and simple posture feedback.
- Users can select basic exercises (squats, plank, arm raises, side bends).

1.5.7 Admin Module

Allow admin to manage users, monitor system statistics, and review feedback.

Features:

- View, edit or delete user accounts.
- Display total registered users, first time posture analysis usage count, first time chatbot usage count, and first time diet plan generation count.
- Review user-submitted feedback comments.
- Update their own personal information.

1.6 Related System Analysis/Literature Review

Many fitness applications are available but most focus only on workouts or only on diet tracking. None of them combine chatbot guidance, BMI based diet plans, and webcam posture correction into a single platform. Table 1.1 compare the PoseFit with other available fitness system.

Table 1: Related System Analysis with proposed project solution

Application Name	Weakness	Proposed Project Solution
Nike Training Club https://www.nike.com/ntc-app	Give exercise videos but not provide diet plan and posture detection and correction.	PoseFit include a diet generator and webcam-based posture detection along with basic workout guidance.
MyFitnessPal https://www.myfitnesspal.com/	Focuses on calorie and nutrition tracking but not provide diet plan and posture detection and correction.	PoseFit combine diet, posture correction, and chatbot in one platform.
Kaia Health https://kaiahealth.de/	Focus mainly on back pain therapy, require paid plans and does not provide diet planning or fitness chatbot support.	PoseFit offers webcam-based posture detection for basic exercises along with an AI-based diet plan generator and fitness chatbot in free web platform.

1.7 Vision Statement

PoseFit is designed for people aged 16 to 50 who want to stay fit at home but do not have access to gyms or personal trainers. It is a web-based fitness assistant that provides a chatbot for fitness guidance, generates personalized diet plans based on Health Metrics, and checks posture during exercises via webcam. Unlike other fitness websites or apps that focus only on workouts or diet, PoseFit combines all three features in one easy-to-use platform, making home fitness simpler, more interactive, and affordable.

1.8 System Limitations and Constraints

Constraints:

1. **Time:** The project has to be finished within the set timeline (Aug 2025 - Jun 2026, as shown in the Gantt chart).
2. **Resources:** The budget is small, and testing is done only with regular webcams and personal laptops.
3. **Technology:** PoseFit is only being developed as a web app. Mobile apps (Android/iOS) are not part of the project.
4. **Accuracy:** The posture checker's performance depends on webcam quality, lighting, and the user's background.
5. **Internet:** A stable internet connection is required for the chatbot, diet planner and posture modules to work properly.
6. **Data Saving:** PoseFit stores only limited user data, including selected health metrics, generated diet plans, limited chat history, and basic posture usage statistics. It does not support saving workout videos or long-term fitness progress tracking.

Limitations:

1. **Exercise Options:** PoseFit only supports a few simple exercises (Squats, Plank, Arm Raises, and Side Bends). It does not cover advanced workouts.
2. **Diet Plans:** The diet plans are limited to three goals (Weight Loss, Muscle Gain, and Maintenance) and are based on Health Metrics. It does not give advanced or medical nutrition advice.
3. **Personal Coaching:** The system does not provide full personalized fitness coaching. The features are basics.
4. **Medical Use:** PoseFit is not a substitute for doctors, physiotherapists, or medical experts. It is only meant for general fitness guidance.

1.9 Tools and Technologies

Table 2: Tools and Technologies for proposed system

Tool/Technology	Version	Rationale
React.js	v19.2	For building a responsive and interactive user interface for the web app.
Tailwind CSS	v4.0	Provides modern, mobile-friendly styling with utility-first classes.
Node.js	20.x LTS	Backend runtime for handling requests and server-side logic.
Express.js	v5.1.0	Lightweight framework to manage API routes and communication.
MongoDB	v8.0	NoSQL database for storing user details (BMI, weight, height, etc.).
TensorFlow.js / MediaPipe	Latest	For real-time webcam posture detection during exercises.
NLP Library / Chatbot API	Latest	To enable chatbot responses for fitness-related FAQs.
Visual Studio Code (VS Code)	1.90+	Primary IDE for coding in JavaScript/React/Node.js.
Git & GitHub	Latest	For version control and maintaining project repository.

Postman	Latest	To test and debug backend APIs during development.
Google Chrome / Firefox	Latest	For testing and running the web application across browsers.

1.10 Project Deliverables

Our system will have the following deliverables by the end of 7th semester:

1. Project Proposal
2. Final Year Project Registration Form
3. Software Requirement Specification (SRS)
4. Software Design Specification (SDS)
5. Prototype
6. Comprehensive Evaluation Document (Proposal, SRS, SDS)

Our system will have the following deliverables by the end of 8th semester:

1. Coding/Implementation (Chatbot, Diet Plan, Posture Detection, UI)
2. Testing
3. Deployment
4. Final Report
5. Presentation & Final Evaluation

1.11 Project Planning

The project planning defines the main tasks and their timelines for developing PoseFit. The Gantt chart shows the schedule, including the duration and sequence of each task from planning to maintenance. This provides a clear overview of the project workflow and deadlines.

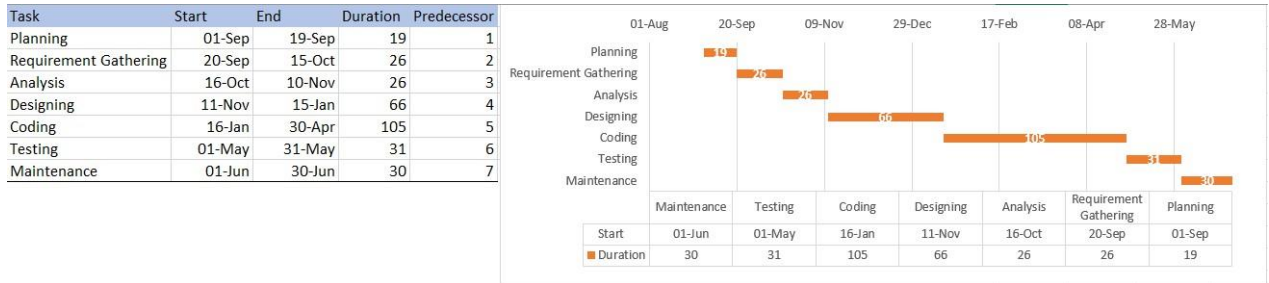


Figure 1: Gantt Chart

1.12 Summary

The PoseFit project is a web-based fitness assistant. It addresses the problem that many people want to stay fit at home but do not have access to a gym or enough time to go to one. The objectives of the system include a chatbot, an AI-based diet planner, and a webcam posture detection tool. The scope of the project and the system components are described, along with the constraints and limitations. The tools and technologies used in the project are also listed, as well as the expected deliverables for the 7th and 8th semesters.

Chapter 2

Requirements Analysis

2. Analysis

PoseFit is a web-based fitness assistant designed to support users in maintaining health and wellness at home. It defines the primary system requirements, user needs, and key features. The focus is on understanding what users want, like fitness guidance, diet plans, and posture checks, and turning these needs into clear system requirements to make PoseFit easy and useful.

2.1 User classes and characteristics

PoseFit is a client-based web application designed to help people stay fit at home. It has two main user types:

Table 3: User class and characterization

User Class	Characterization
End User	People aged 16-50 who use PoseFit on mobile or laptop to chat with the fitness bot, get diet plans, and check their exercise posture using a webcam. They need a simple and easy-to-use interface.
Admin	The administrator manages users, system data, and user feedback using a separate admin panel to keep the system working properly and secure.

2.2 Requirement Identifying Technique

The techniques used to identify and define the functional requirements of the PoseFit system are determined by the nature of the project and the level of the user interaction involved. Given that PoseFit is an interactive web-based application with multiple user interactions, use case-based requirement analysis is used to clearly understand system behavior and user actions.

The use case model includes:

- **Use Case Description:** It provides detailed explanation of each function, including normal flow, alternative flows, and system rules.

- **Use Case Diagram:** It provides a visual overview of the system, showing actors and their interactions with major features.

2.3 Functional Requirements

The PoseFit system outlines its functionality and operations for users through three main functions: a chatbot, an AI-based diet plan generator, and posture detection. Each function is described below as a separate functional requirement:

2.3.1 Functional Requirement 1

Table 4: FR-1

Identifier	FR-1
Title	User Sign Up with Email Verification
Requirement	The system shall allow users to create a new account using an email and password, and activate it after email verification
Source	Project requirement
Rationale	To ensure that only valid users can register and use the system.
Business Rule	Password must be at least 8 characters, have one uppercase letter, one digit, and one special character.
Dependencies	Database connection, Email service, Internet access.
Priority	High

2.3.2 Functional Requirement 2

Table 5: FR-2

Identifier	FR-2
Title	Login
Requirement	The system shall allow users and administrators to log in using a valid email and password and access features based on their role, including logging out.
Source	Project requirement
Rationale	Provide secure environment.
Business Rule	User must be registered before sign in.
Dependencies	Valid registered user data in the database, Active internet connection,
Priority	High

2.3.3 Functional Requirement 3

Table 6: FR-3

Identifier	FR-3
Title	Password Reset
Requirement	The system shall allow users to reset their password by sending a reset link to their registered email.
Source	Project requirement.

Rationale	To help the users recover their accounts if they forget their password.
Business Rule	New passwords must be at least 8 characters long, include letters and numbers, and cannot reuse old passwords.
Dependencies	FR-1
Priority	Medium

2.3.4 Functional Requirement 4

Table 7: FR-4

Identifier	FR-4
Title	Google Sign-In/Up
Requirement	The system shall allow users to sign up or log in using their Google account instead of entering an email and password.
Source	Project enhancement
Rationale	To allow quick and secure access without creating a new password.
Business Rule	The system shall access only users' name and email from Google and not store any sensitive data.
Dependencies	Google Authentication API, Active internet connection.
Priority	Medium

2.3.5 Functional Requirement 5

Table 8: FR-5

Identifier	FR-5
Title	Fitness Chatbot
Requirement	The system shall allow users to chat with the fitness bot to get quick FAQ'S about workout.
Source	User feedback and observation.
Rationale	To provide user with quick fitness support without searching online.
Business Rule	The chatbot shall provide clear and relevant answers to users' queries. The chatbot shall save a limited history of user's interactions.
Dependencies	Internet connection, Chatbot API
Priority	High

2.3.6 Functional Requirement 6

Table 9: FR-6

Identifier	FR-7
Title	Health Metrics

Requirement	The system shall allow users to enter personal information, select activity level and goal, calculate BMI, BMR, TDEE, and Macros, and display the results.
Source	Project Requirement
Rationale	To help users understand their body metrics and daily calorie requirements for informed health decisions.
Business Rule	- BMI shall be calculated based on weight and height. - BMR shall be calculated based on gender, age, height, and weight. - TDEE shall be adjusted according to the selected activity level. - Macronutrients shall be adjusted based on the user's fitness goal. - All numeric input fields (age, height, weight) must contain valid values before calculation
Dependencies	User interface form inputs (gender, age, height, weight), Activity Level Selection
Priority	High

2.3.7 Functional Requirement 7

Table 10: FR-7

Identifier	FR-7
Title	Diet Plan Generator

Requirement	The system shall allow users to generate a personalized diet plan based on their Health Metrics and preferences, providing daily intake of protein, carbohydrates, fats, and calories.
Source	Project requirement.
Rationale	To help users follow a goal-based diet that aligns with their energy requirements and fitness objectives.
Business Rule	- The diet plan shall be based on the user's Health Metrics and preferences. - Macronutrient distribution shall follow standard ratios but may be adjusted according to the user's goal. - All recommended daily intake values shall be positive and realistic.
Dependencies	FR-6
Priority	Medium

2.3.8 Functional Requirement 8

Table 11: FR-8

Identifier	FR-8
Title	Posture Detection
Requirement	The system shall detect the user's posture using a webcam and show feedback if the exercise posture is correct or not.

Source	Observation and system testing
Rationale	To help users correct their posture and avoid exercise mistakes.
Business Rule	The system shall process posture data and give clear feedback.
Dependencies	Webcam access, Media Pipe API for key-points detection, Internet connection.
Priority	High

2.3.9 Functional Requirement 9

Table 12: FR-9

Identifier	FR-9
Title	Profile Management
Requirement	The authenticated user can view and update their personal profile information.
Source	Project Requirement
Rationale	To allow users to manage their personal data.
Business Rule	User must provide a valid email address, first name and last name cannot be empty.
Dependencies	Database connection, user must be logged in.
Priority	High

2.3.10 Functional Requirement 10

Table 13: FR-10

Identifier	FR-10
Title	Submit Feedback
Requirement	The system shall allow the users to submit feedback regarding their experiences with the application.
Source	Project Requirement
Rationale	To collect user opinion and suggestions for system improvement.
Business Rule	Feedback text field must not be empty.
Dependencies	Database connection, User must be logged in
Priority	Medium

2.3.11 Functional Requirement 11

Table 14: FR-11

Identifier	FR-11
Title	Logout
Requirement	The system shall allow all users, including admin, to securely log out from their account or dashboard.
Source	Security Requirement
Rationale	To insure account security and prevent unauthorized access.

Business Rule	User and admin sessions must be destroyed immediately after logout.
Dependencies	Authentication system, session management
Priority	High

2.3.12 Functional Requirement 12

Table 15: FR-12

Identifier	FR-12
Title	View Dashboard Analytics
Requirement	The system shall display overall statistics including total users, diet plans generated, and posture usage count.
Source	Admin Requirement
Rationale	This helps the admin easily monitor system usage and overall performance.
Business Rule	The data will be updated and fetched from the database.
Dependencies	Database connection, Internet access
Priority	High

2.3.13 Functional Requirement 13

Table 16: FR-13

Identifier	FR-13
Title	User Details Management
Requirement	The system shall allow the admin to view, update, or delete registered user accounts.
Source	System Maintenance Requirement
Rationale	To manage users and maintain system integrity.
Business Rule	Admin confirmation required before deleting a user.
Dependencies	Database, Admin Authentication, Internet access
Priority	High

2.3.14 Functional Requirement 14

Table 17: FR-14

Identifier	FR-14
Title	Feedback Review
Requirement	The system shall allow the admin to review feedback submitted by user.
Source	Project Requirement

Rationale	To improve system features and user experience.
Business Rule	Feedback is read-only and cannot be modified.
Dependencies	Database connection, Internet access
Priority	Medium

2.4 Non-Functional Requirements

The quality standards of PoseFit define requirements that ensure the system is reliable, user-friendly, secure, and performs efficiently under normal conditions.

2.4.1 Reliability

PoseFit shall operate smoothly without frequent errors or system crashes. User data such as health metrics, diet plans, and chat history shall remain available and consistent whenever the user accesses the system.

2.4.2 Usability

The system shall be easy to understand and use for all users. Navigation between features such as chatbot, fitness metrics, diet plans, and posture detection shall be clear and simple, requiring minimal learning effort.

2.4.3 Performance

PoseFit shall provide quick responses during normal usage. Chatbot replies and diet plan generation shall complete within a few second, and posture detection shall run smoothly without noticeable delays.

2.4.4 Security

User data shall be protected using secure authentication methods. Passwords shall be stored securely, and users shall only be able to access their own data. Admin access shall be restricted and protected from unauthorized use.

2.4.5 Portability

PoseFit shall be accessible on major web browsers such as Google Chrome and Microsoft Edge. The system shall function properly on both desktop and mobile devices without requiring additional installation.

2.4.6 Maintainability

The system code shall be modular and easy to maintain. Developers shall be able to update features such as chatbot responses or diet Plan login without affecting other parts of the system.

2.4.7 Scalability

PoseFit shall support future growth. The system design shall allow an increase in users and the additional of new features, such as more exercises or advanced diet options, without major changes to the existing system.

2.5 External Interface Requirement

The external interfaces of the PoseFit system specify how the system interacts with users, hardware devices, and external software components. PoseFit is a web-based fitness application that allows users to interact with the system through a graphical user interface (GUI). The system also communicates with external software services to provide its features.

PoseFit does not require any specialized hardware. However, it requires standard input and output devices such as a keyboard, mouse, display screen, and webcam for user interaction and posture detection. The system uses external software interfaces for user authentication, chatbot communication, posture detection processing, and database management. These interfaces are

designed to provide secure, reliable, and efficient communication between the PoseFit system and external services.

2.5.1 User Interface Requirements

The user interface of the PoseFit system is designed to be clear, consistent, and easy to use for all users.

The PoseFit system follows standard web-based graphical user interface (GUI) guidelines to ensure a consistent user experience.

The user interface meets the following standards:

- The system uses clear and readable sans-serif font.
- Standard icons are used for navigation and common actions.
- Button labels are simple, clear, and action based.
- A consistent color scheme suitable for a fitness application is used.
- Common interface controls such as buttons, menus, and input fields are consistent across all screens
- Logical tab order is maintained for input fields to support keyboard navigation.

The system supports the following layout requirements:

- The interface supports both desktop and mobile devices.
- The minimum supported screen resolution is 360 x 640 pixels.
- The layout adjusts automatically to different screen sizes.

The following standard interface elements are available where applicable:

- Navigation link such as Home, Profile, and Logout.
- Standard actions buttons (e.g., Submit, Save, Cancel).

The system follows these interaction and message display rules:

- All input fields support keyboard input.
- Success, error, and informational messages are displayed clearly.
- Messages use simple and understandable language.

Accessibility considerations:

- The text is clearly visible against the background.

- Interface elements are large enough for easy interaction.
- The interface supports users with minor visual impairments.
- The interface layout allows future localization of the software.

2.5.2 Software Interfaces

The PoseFit system interacts with external software components to support its functionality. **These include:**

- Web browsers such as Google Chrome and, Edge, and Firefox.
- Authentication services for users login and registration.
- Chatbot services for user interaction.
- Database services for storing user data, diet plans, chat history, and feedback.

All software interfaces use secure and reliable web-based communication methods.

2.5.3 Hardware Interfaces

The PoseFit system does not require specialized hardware. It interacts with standard hardware devices, **including:**

- Keyboard and mouse for user input.
- Display screen for visual output
- Webcam for posture detection

The webcam is accessed during standard browser-based APIs.

2.5.4 Communication Interfaces

The PoseFit system uses standard web communication technologies.

These include:

- HTTP and HTTPS protocols for data transmission.
- Browser-based communication for user interaction.
- Secure network connections to protect user data

No additional communication interfaces such as email or SMS are required in the current version of the system.

2.6 Use Case Analysis

All systems include some kind of interaction. In PoseFit, interaction happens mainly between the user and the web system. Users give input through the chatbot, diet plan, and posture detection tools, and the system provides suitable responses or results. User interaction modeling is important because it helps to understand what users need and how they will use the system. It also makes sure that all features, like chatting, saving data, and posture checking, work smoothly together.

2.6.1 Use Case 1

Table 18: UC-1

Use Case ID	UC-1
Use Case Name	User Registration
Actors	Guest User
Description	This use case allows a new user to create an account in the PoseFit application.
Trigger	User opens the app and selects the "Register" option.
Pre-conditions	User is not already registered.
Post-conditions	User account is created and email is verified.

Normal Flow	- User opens the registration page. - User enters email, password, and personal details. - System sends a verification code to the user's email. - User enters the verification code. - System verifies the code. - System creates the user account. - System shows a success message and redirects to login page.
Alternative Flows	Invalid Code: System shows an error message and asks user to try again. Expired Code: System sends a new verification code.
Business Rules	- Email must be unique and valid. - Password must have at least 8 characters, one uppercase, one number and one special character.
Assumptions	- User has internet access. - System can send verification emails.

2.6.2 Use Case 2

Table 19: UC-2

Use Case ID	UC-2
Use Case Name	User Login
Actors	Registered User
Description	Allows a registered user to log in.

Trigger	User selects "Login".
Preconditions	User must already be registered.
Postconditions	User is logged in successfully.
Normal Flow	- User opens login page. - User enters email and password. - System checks credentials. - System creates a secure session. - User is redirected to dashboard.
Alternative Flows	Invalid Credentials - System shows error. Google Login - User selects Google, system authenticates.
Business Rules	- Login requires registered email and correct password. - JWT token expires after a set time.
Assumptions	- User knows credentials. - Google OAuth is configured.

2.6.3 Use Case 3

Table 20: UC-3

Use Case ID	UC-3
Use Case Name	Password Reset
Actors	Registered User

Description	Allows user to reset forgotten password.
Trigger	User clicks "Forgot Password".
Preconditions	User has a registered email.
Postconditions	Password updated successfully.
Normal Flow	- User clicks "Forgot Password". - User enters registered email. - System sends reset link. - User clicks link. - User enters new password. - System updates password. - System shows success message.
Alternative Flows	Invalid Link: User clicks an invalid reset link - System shows an error and prompts to request a new link. Expired Link: User clicks an expired reset link - System sends a new reset link and notifies the user.
Business Rules	- Reset link expires after a set time. - New password must follow password policy.
Assumptions	- User has access to email, system can send emails.

2.6.4 Use Case 4

Table 21: UC-4

Use Case ID	UC-4
Use Case Name	Chatbot
Actors	Logged-in User
Description	Allows user to ask fitness questions using chatbot.
Trigger	User opens Chatbot page.
Preconditions	User must be logged in.
Postconditions	Conversation is saved and displayed.
Normal Flow	- User opens Chatbot page. - User types question. - System sends question to AI. - AI generates response. - System saves conversation in database. - System displays response.
Alternative Flows	User not logged in: User tries to ask a question without login - System prompts user to log in first. AI unavailable: AI model fails to respond - System shows an error Message and suggests trying later. System save failure: Database fails to save conversation - System shows warning that chat may not be saved.

Business Rules	- Only logged-in users can ask queries. - Each query is saved with timestamp.
Assumptions	- AI service/API is available for generating chatbot responses. - User types questions in a supported language.

2.6.5 Use Case 5

Table 22: UC-5

Use Case ID	UC-5
Use Case Name	Health Metrics
Actors	Logged-in User
Description	User enters personal info to calculate BMI, BMR TDEE, and Macros without generating diet plan.
Trigger	User has completed health metrics calculation and opens diet plan section.
Preconditions	User logged in.
Postconditions	Calculated BMI, BMR, TDEE, and Macros displayed.
Normal Flow	- User opens Diet Plan page. - Enters gender, age, height, weight, activity level, and goal. - System calculates BMI, BMR, TDEE, and Macros. - System displays results.
Alternative Flows	Invalid or empty input: Show error and highlights fields.

Business Rules	Inputs must be valid (e.g., numbers, positive)
Assumptions	User provides accurate information.

2.6.6 Use Case 6

Table 23: UC-6

Use Case ID	UC-6
Use Case Name	Generate Diet Plan
Actors	Logged-in User
Description	Generates a personalized diet plan based on user's Health Metrics and preferences.
Trigger	User opens Diet Plan page.
Preconditions	User must be logged in and has calculated Health Metrics.
Postconditions	Diet plan is generated and saved in the database for future reference.
Normal Flow	- System fetches calculated Health Metrics from the system. - System asks follow-up preference questions. - System generates a personalized diet plan based on the saved metrics and user preferences. - Generated diet plan is displayed to the user.

	- Diet plan is saved in the database.
Alternative Flows	- Health Metrics not found system prompts: “Please calculate your Health Metrics first”. - Database save failure: If the system fails to save the diet plan, a warning message is displayed: “Diet plan may not be saved, please try again.”
Business Rules	Diet plan must be based on user’s metrics and goal.
Assumptions	User have already calculated BMI.

2.6.7 Use Case 7

Table 24: UC-7

Use Case ID	UC-7
Use Case Name	Posture Detection
Actors	Logged-in User
Description	This use case allows user to detect posture using webcam.
Trigger	User opens Posture Detection page.
Preconditions	Webcam must be available.
Postconditions	Posture is analyzed and data is not saved.
Normal Flow	- User opens posture page.

	- User selects exercise. - System asks for webcam permission. - User allows permission. - System starts posture model. - User performs exercise. - System analyzes posture. - System display feedback. - User may stop or switch exercise.
Alternative Flows	Webcam permission denied: System shows error message. Exercise not selected: System asks user to choose an exercise.
Business Rules	- Only logged-in users can use posture detection. - Webcam access is required. - Analysis results are shown immediately and not stored.
Assumptions	- User has a working webcam. - Posture model is available and works correctly.

2.6.8 Use Case 8

Table 25: UC-8

Use Case ID	UC-8
Use Case Name	Update Profile
Actors	Authenticated User
Description	This use case allows the user to view and update their profile settings such as name, email and password.

Trigger	User opens the profile settings page.
Preconditions	User must be logged in.
Postconditions	User profile is updated successfully.
Normal Flow	- User opens the profile settings page. - System displays current profile information. - User edits profile fields (e.g., name, email, password) - User clicks the save/update button. - System validates the input and updates the database. - System confirms successful update to the user.
Alternative Flows	- User enters invalid data. - System displays an error message and prompts correction.
Business Rules	- Mandatory fields (like email) must be filled. - Password changes must follow security rules. - Changes are saved only after validation.
Assumptions	- Profile settings page is available in the system. - User data exists in the database. - Database is working properly.

2.6.9 Use Case 9

Table 26: UC-9

Use Case ID	UC-9
Use Case Name	Submit feedback

Actors	User
Description	This use case allows the user to submit feedback about the system.
Trigger	User opens the feedback form.
Preconditions	User must be logged in.
Postconditions	Feedback is successfully stored in the database.
Normal Flow	- User opens the feedback page. - System displays the feedback form. - User writes feedback text. - User clicks the submit button. - System validates the feedback. - System stores feedback in the database.
Alternative Flows	- User submits empty feedback. - System displays an error message asking for required text.
Business Rules	Feedback text is mandatory and cannot be empty.
Assumptions	- Feedback form is available. - Database is working properly.

2.6.10 Use Case 10

Table 27: UC-10

Use Case ID	UC-10
Use Case Name	View Dashboard Analytics
Actors	Admin
Description	This use case allows the admin to view system-wide statistics and analytics on the dashboard.
Trigger	Admin opens the dashboard page.
Preconditions	Admin must be logged in.
Postconditions	System statistics are successfully displayed on the dashboard.
Normal Flow	- Admin opens the dashboard. - System fetches data from the database. - System displays statistics such as total users, total diet plans generated count, posture usage count.
Alternative Flows	Database error occurs system displays an error message indicating data cannot be loaded.
Business Rules	The dashboard should show the latest system data
Assumptions	- Dashboard is accessible to the admin. - Required data exists in the database. - Database connection is working properly.

2.6.11 Use Case 11

Table 28: UC-11

Use Case ID	UC-11
Use Case Name	Manage Users
Actors	Admin
Description	This use case allows the admin to view, update, or delete user accounts in the system.
Trigger	Admin opens the user management page.
Preconditions	Admin must be logged in.
Postconditions	User information is updated or removed successfully.
Normal Flow	- Admin opens the user management page. - System displays a list of users. - Admin selects a user. - Admin chooses an action (view, update, or delete). - System asks for confirmation. - System performs the selected action and updates the database.
Alternative Flows	- Admin cancels the selected action. - No changes are made to user data.
Business Rules	Confirmation is required before updating or deleting any user data.

Assumptions	- User data exists in the system. - Admin has permission to manage users. - Database is working properly.
--------------------	---

2.6.12 Use Case 12

Table 29: UC-12

Use Case ID	UC-12
Use Case Name	Review Feedback
Actors	Admin
Description	This use case allows admin to review user feedback.
Trigger	Admin opens feedback page.
Preconditions	Admin is logged in and feedback exists.
Postconditions	Feedback is reviewed.
Normal Flow	- Admin opens feedback page. - System shows feedback list. - Admin reads feedback. - Admin marks feedback as “reviewed”.
Alternative Flows	If no feedback is available, system shows “No feedback yet.”
Business Rules	- Only admin can mark feedback as “reviewed”. - Feedback cannot be changed, only marked reviewed.

Assumptions	- Feedback exists in the system. - Admin can see which feedback is already reviewed. - Database is working properly.
--------------------	--

2.7 Use Case Diagram

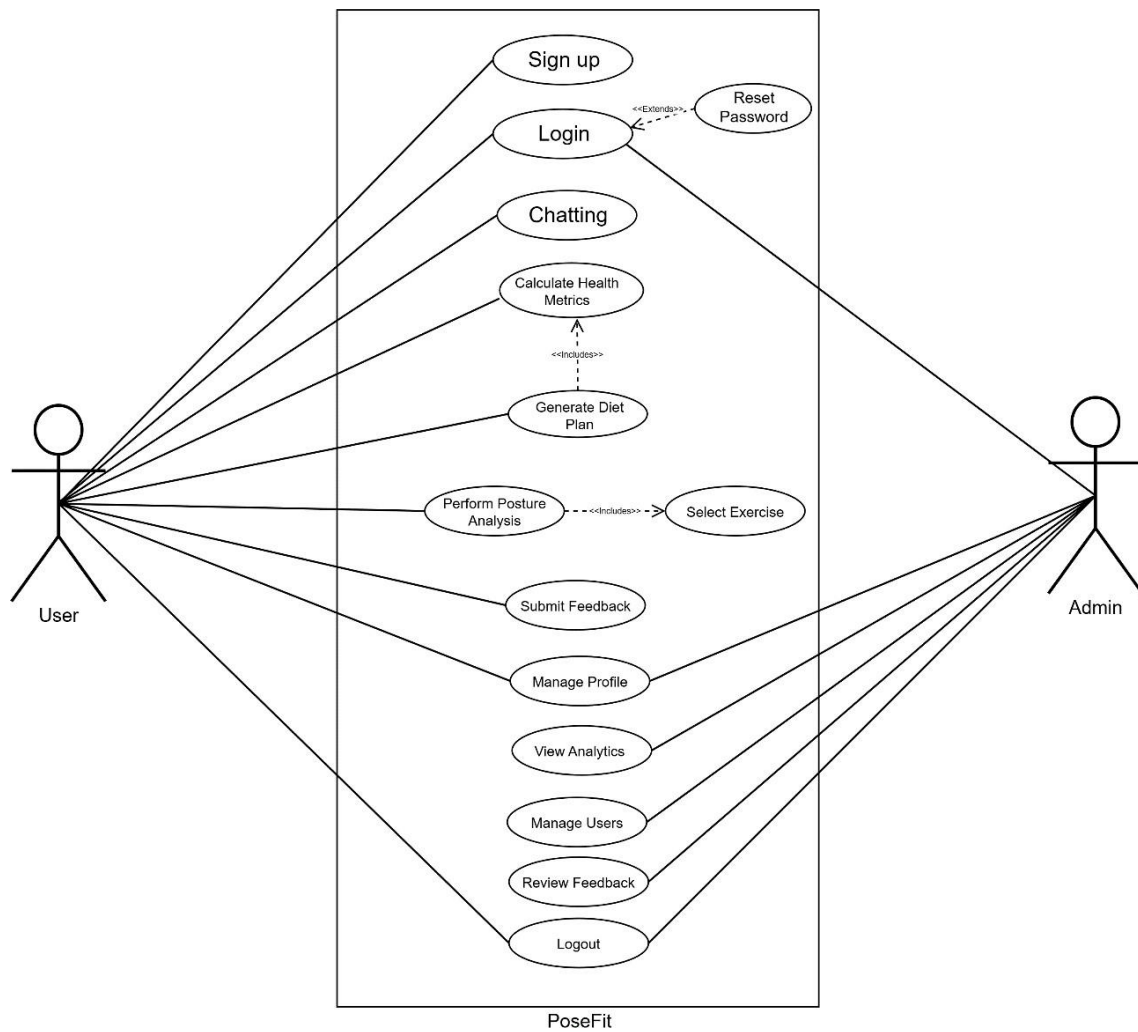


Figure 2: Use Case Diagram

2.8 Summary

An analysis of the PoseFit system and its requirements is presented. It covers the functional requirements such as authentication, chatbot interaction, health metrics calculation, diet plan generation, and posture detection. The non-functional requirements including usability, performance, security, and reliability are also discussed. System components, external interfaces, and use cases are defined to explain how the system works and interacts with users and hardware.

Chapter 3 Design and Architecture

3. System Design

The system design of PoseFit is presented, covering its structure, operation, and interaction with users. High-level models such as class, sequence, state and component diagrams are used to represent the system. The application follows the MVP (Model-View-Presenter) architecture for clear separation of concerns, ease of understanding, and testability, supporting features like posture detection, diet planning and chatbot interactions.

3.1 Design considerations

Key factors influencing the design of the PoseFit system, including assumptions, limitations, and potential risks are described.

3.1.1 Assumptions and Dependencies

The following assumptions were made during the design of PoseFit:

- Users have a stable internet connection.
- Users have devices with a working webcam for posture detection.
- Users access the system through modern web browsers that support JavaScript and camera access.
- User provide accurate personal and health information (age, weight, height, goals).
- Users perform exercises from the predefined set supported by the system.
- Admin users have basic technical knowledge to manage the system.

The system depends on several external components:

- MediaPipe library for posture detection and landmark extraction.
- Pre-trained chatbot models for fitness-related queries.
- Web browser APIs for camera access and interaction.
- Backend services for data storage and processing, model inference, and diet plan generation.

These dependencies directly affect system functionality and performance.

3.1.2 Limitations

The PoseFit system has some limitations:

- Posture detection accuracy may vary with lighting conditions and camera quality.
- Only a limited number of exercises are supported in the current version.
- Chatbot responses are restricted to fitness related queries and predefined knowledge.
- Processing may be slower on low-performance devices or with high- resolution input.
- The system provides general fitness guidance and does not offer medical advice.
- Posture analysis is based on the trained model and may not account for all variations in user movements.
- System functionality depends on stable internet connectivity, and some features may vary slightly across different web browsers.

These limitations define the system boundaries and set realistic user expectations.

3.1.3 Risks

Table 30: Risks and Mitigation Strategy

Risk	Mitigation strategy
Inaccurate posture detection.	Use tested AI libraries and a controlled exercise set.
Performance lag during analysis.	Optimized frame processing and lightweight models.
Data privacy concerns.	Implement secure authentication and restricted data access.
Dependency failure (AI libraries or APIs).	Use stable versions and fallback mechanisms.

3.2 Design Models

Design models in the poseFit system describes the system structure, behavior, and interactions.

These models help transform system requirements into a clear and organized software design.

PoseFit uses a Class Diagram to represent system classes and their relationships, a Sequence Diagram to show user system interactions during key processes such as diet plan generation and posture detection, and a State Transition Diagram to illustrate changes in user or system states like login and chatbot interaction. Together, these models ensure that the system design is clear, modular, and easy to maintain.

3.2.1 Class Diagram

The class diagram is a structural UML diagram modeling the PoseFit system using Object-Oriented Programming principles, with each major component represented as a class. Both regular users and administrators are modeled under a single User class, distinguished by a role attribute (e.g., "user" or "admin"), simplifying authentication and access control. The diagram includes classes like User, Chatbot, DietPlan, HealthMetrics, PostureDetection, Feedback, and AdminService, with their key attributes and methods. Relationships are mainly associations: a User can generate DietPlans, submit Feedback, interact with the Chatbot, and use PostureDetection, while AdminService manages user accounts and reviews feedback for admin roles. This design ensures modularity, reduces redundancy, and fits the MERN stack's document-based model.

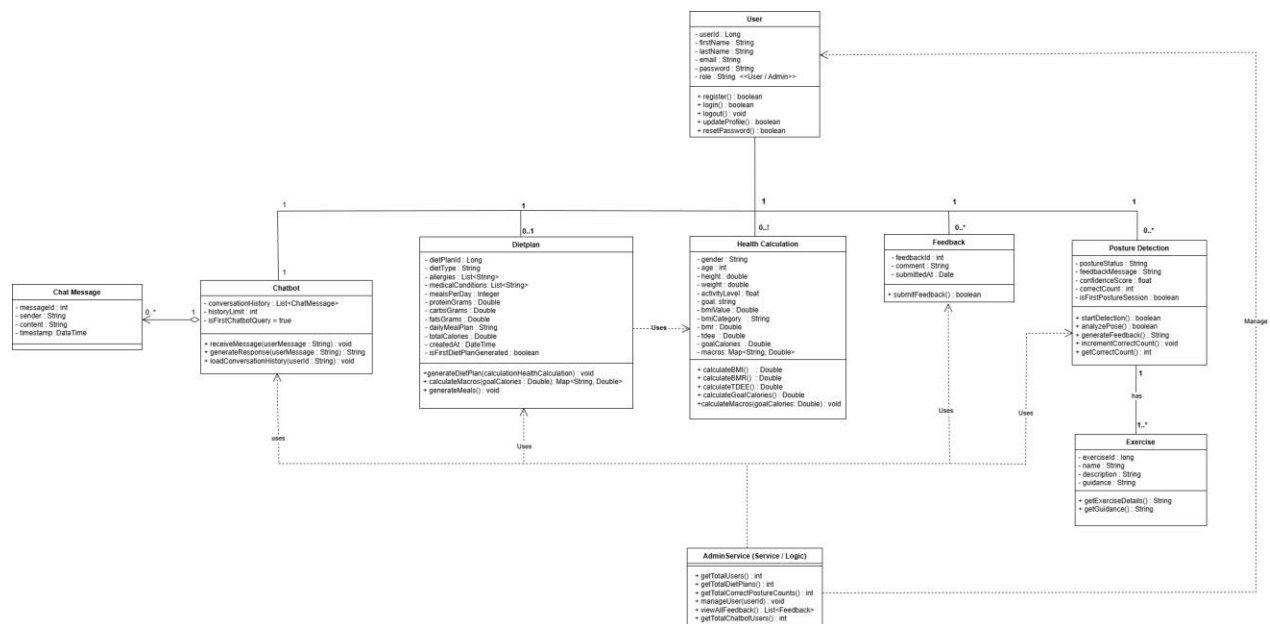


Figure 3: Class Diagram

3.2.2 Sequence Diagram

These sequence diagrams comprehensively model the core user journeys in the system, from account registration and login (including Google OAuth and password recovery) to personalized diet planning, instant posture feedback during exercises, and AI-powered chatbot use. The other diagrams show feedback submission and admin moderation processes, covering all user features and backend management. Together, they explain message flows, conditional steps, and responsibilities of frontend, backend, and external services, providing a clear view of system behavior and integration.

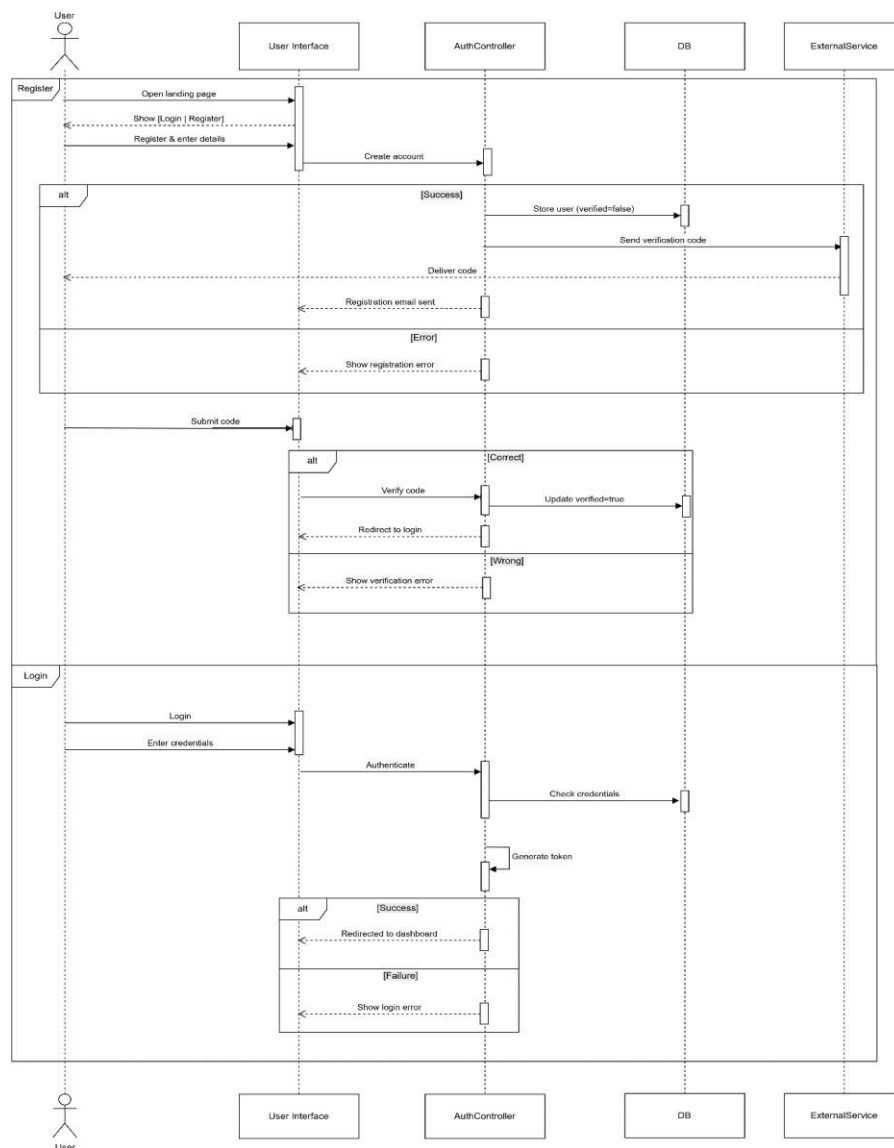


Figure 4: Sequence Diagram 1

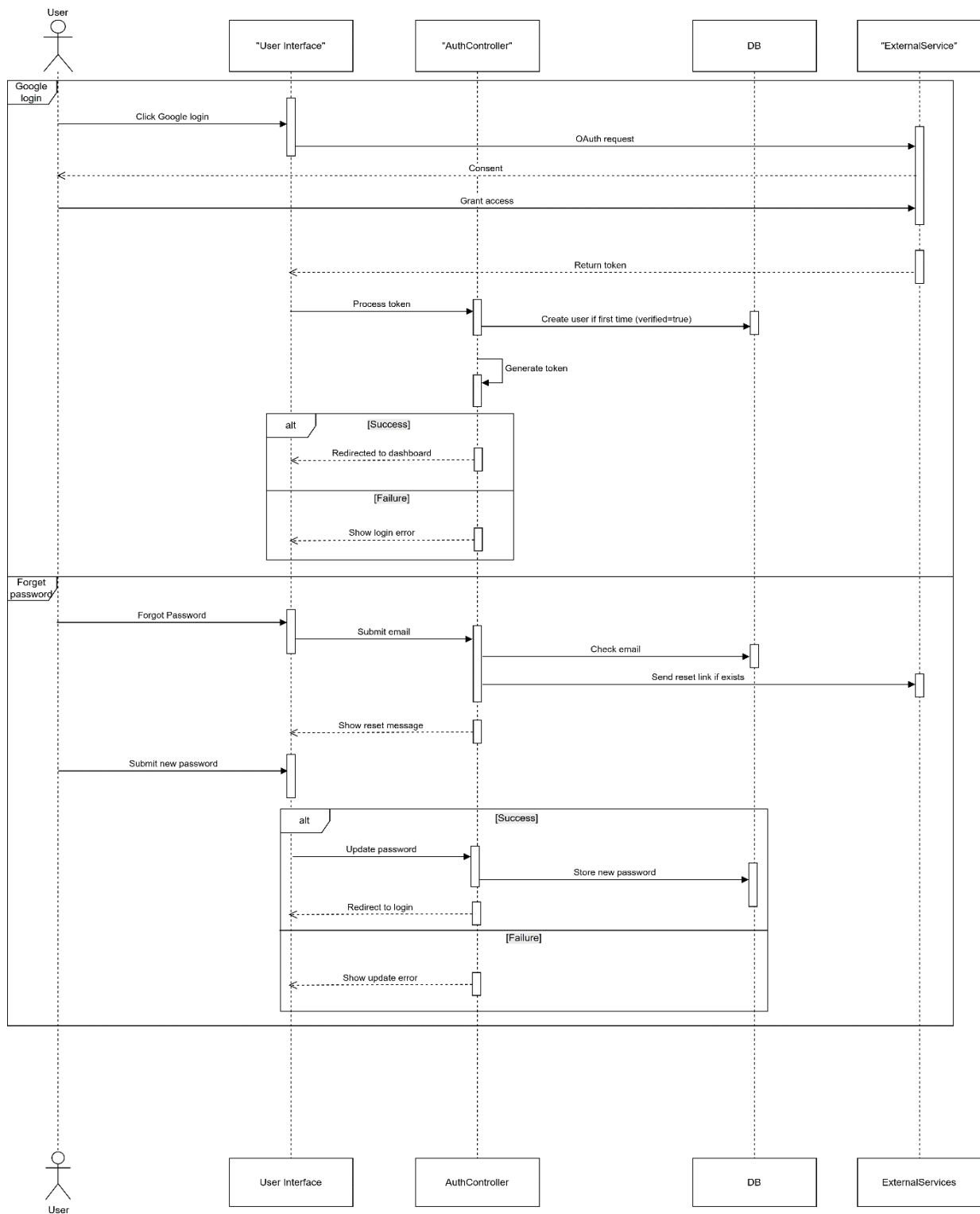


Figure 5: Sequence Diagram 2

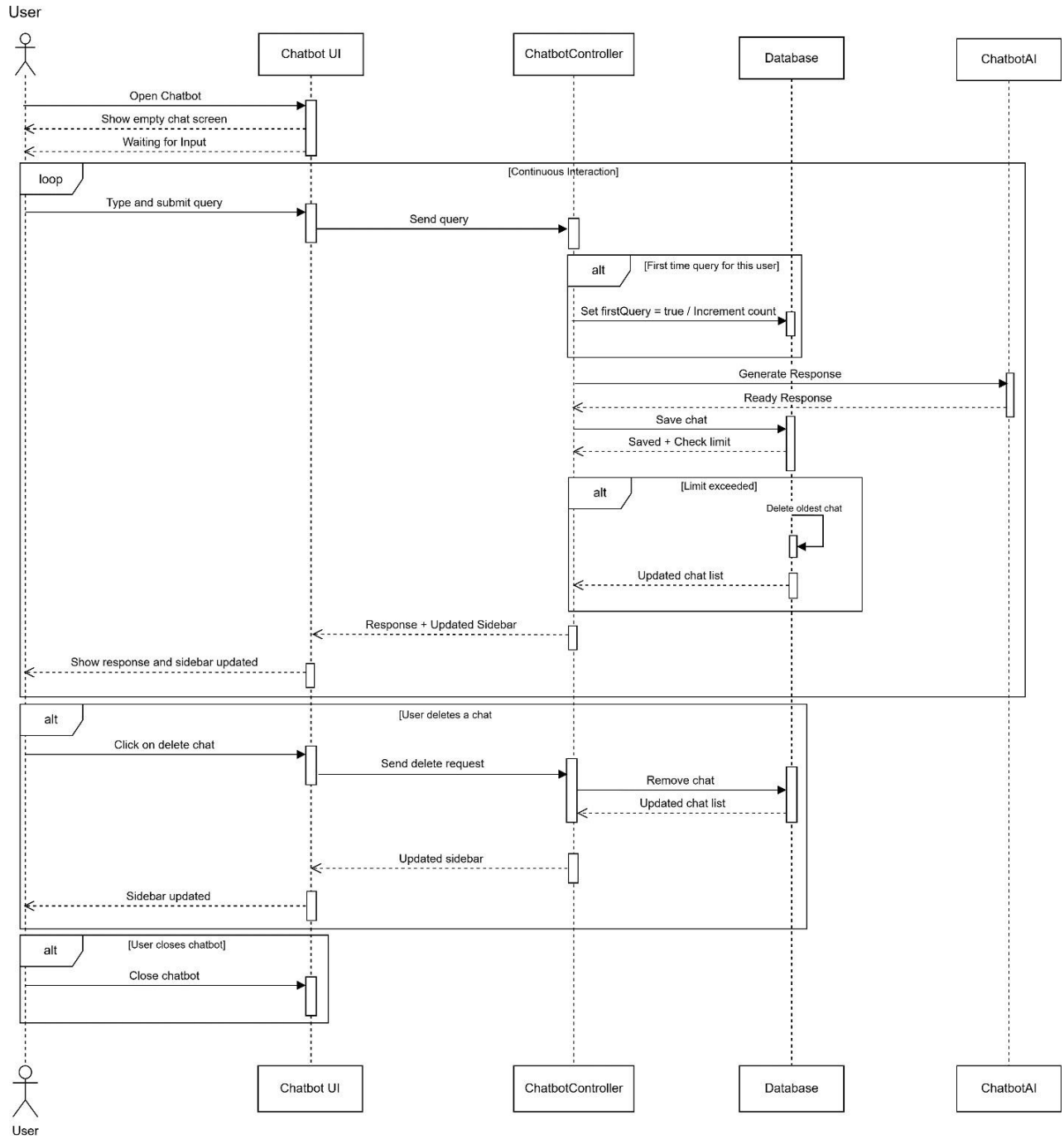


Figure 6: Sequence Diagram 3

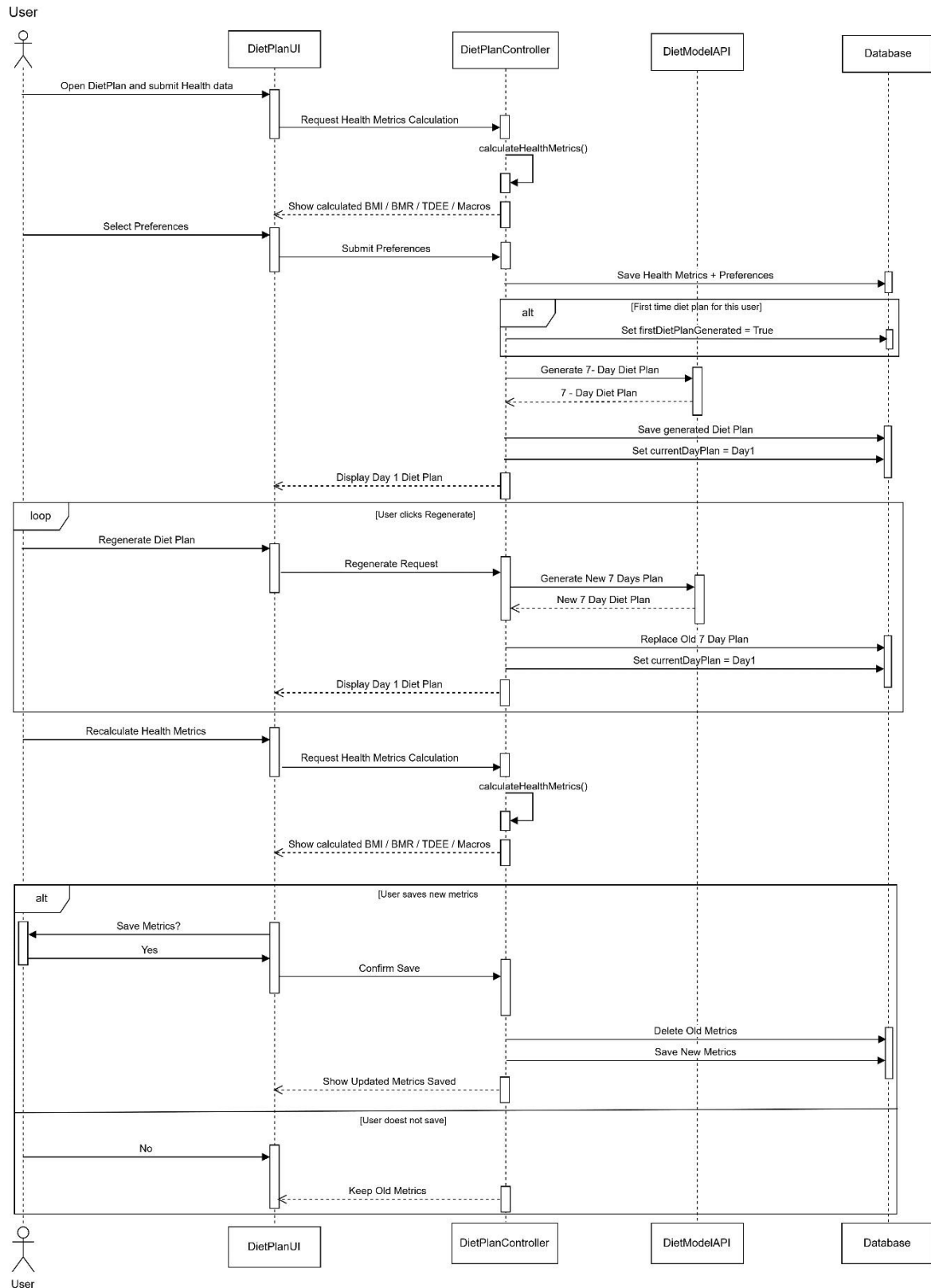


Figure 7: Sequence Diagram 4

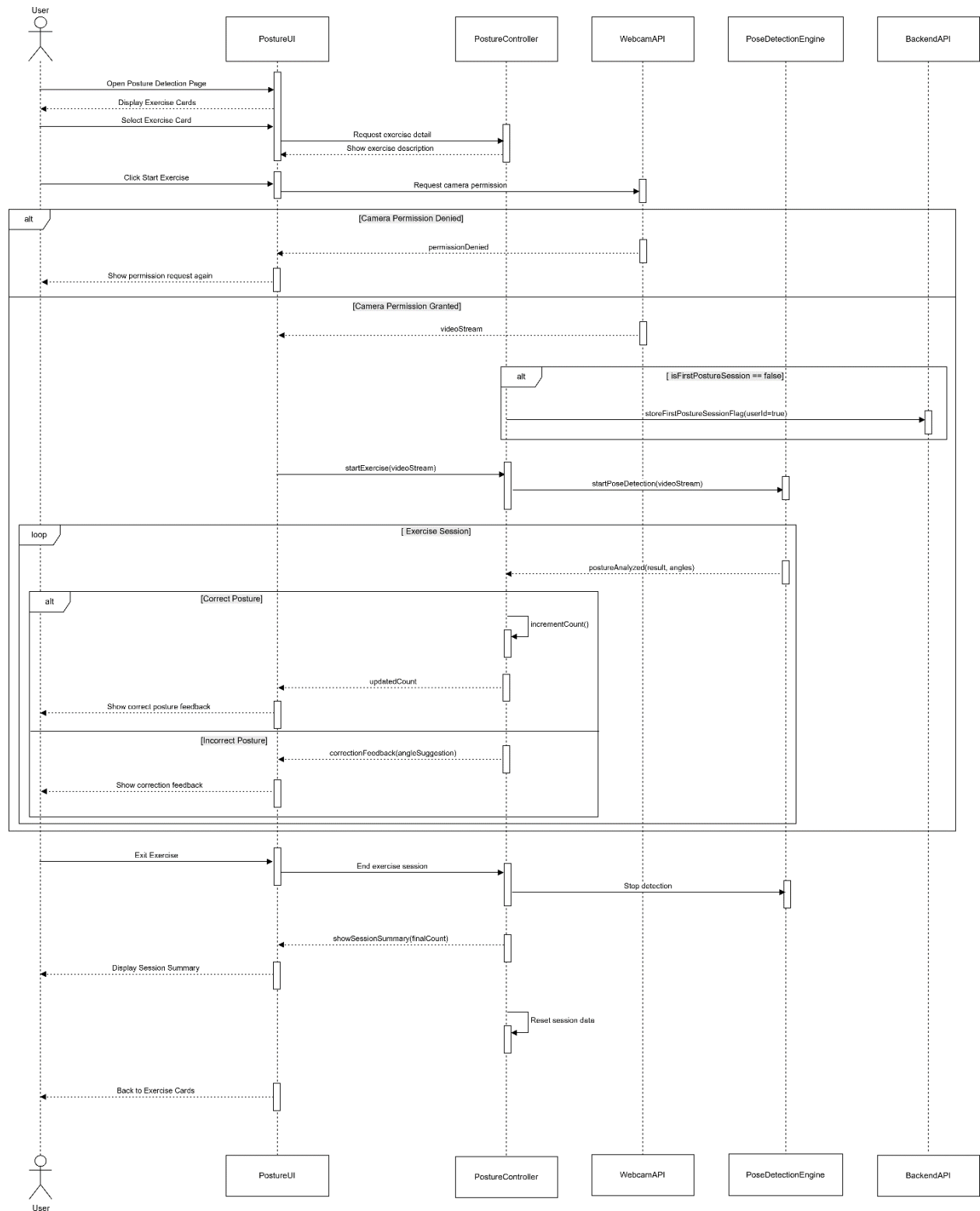


Figure 8: Sequence Diagram 5

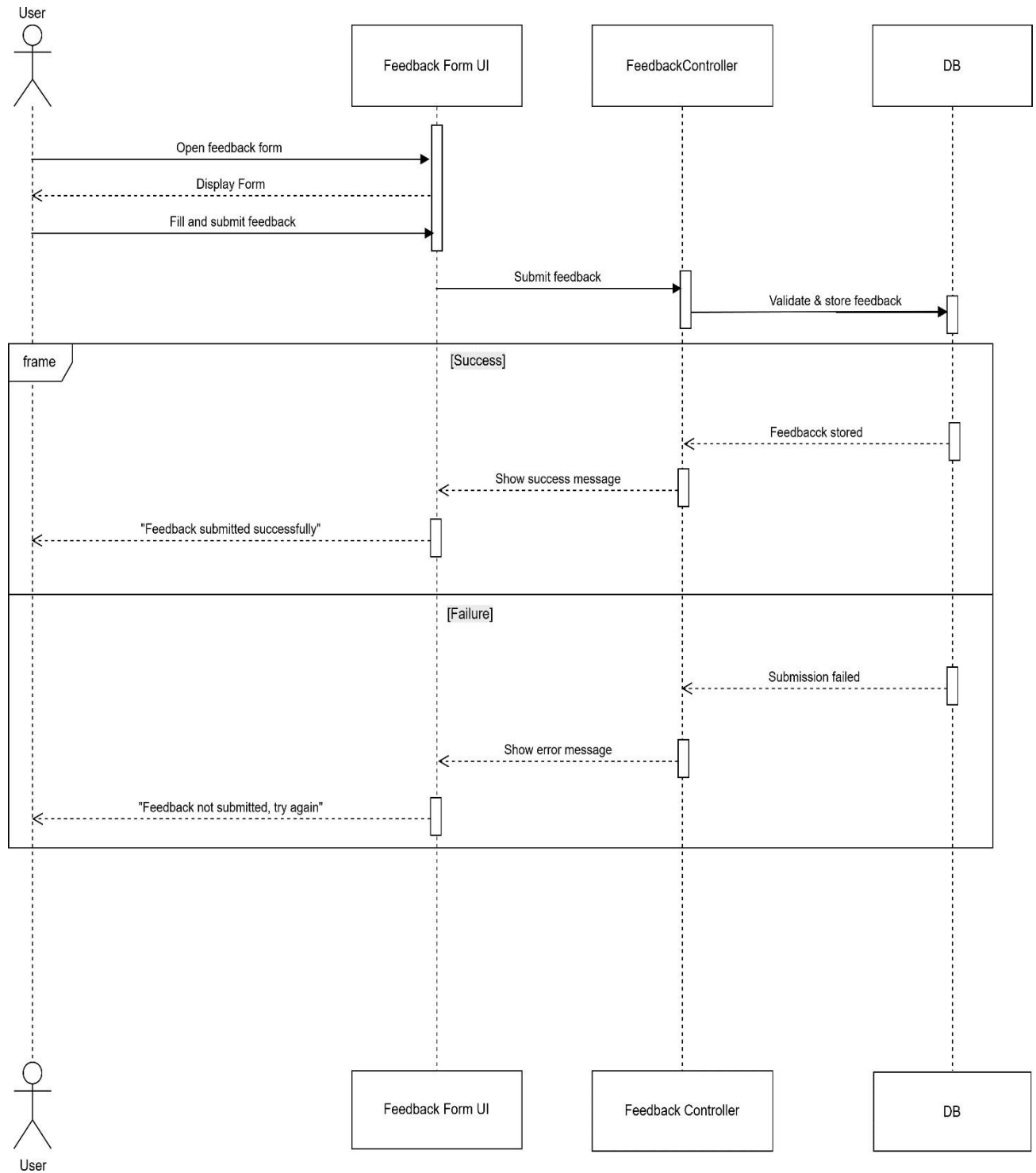


Figure 9: Sequence Diagram 6

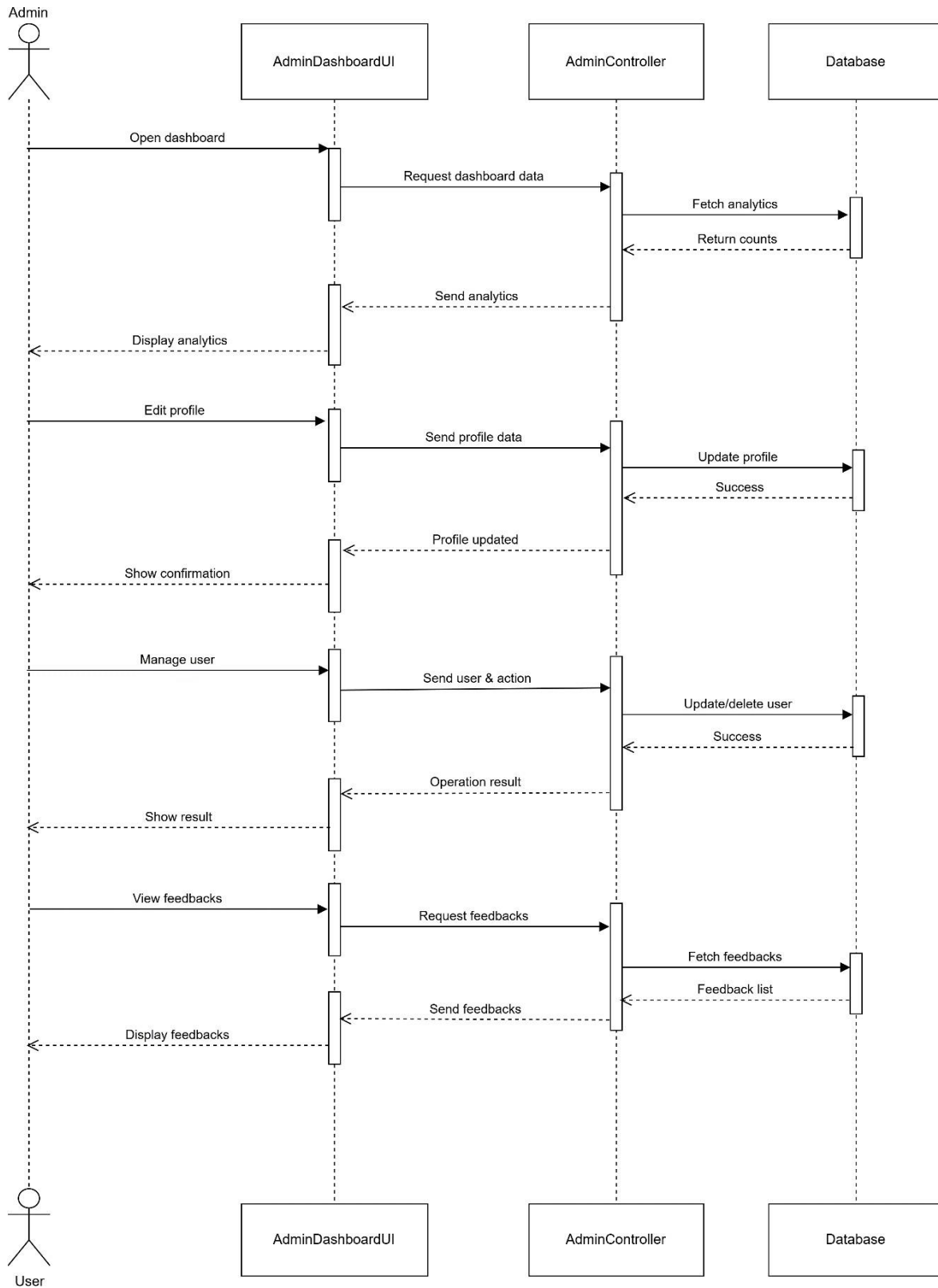


Figure 10: Sequence Diagram 7

3.2.2 State Transition Diagram

The state transition diagrams illustrate the lifecycle of key user workflows from unauthenticated access through registration, login (email/Google), and password recovery to reaching the Dashboard. For core features like Chatbot, Diet Plan, and Posture Detection, each diagram maps valid state transitions capturing user actions, system responses, success/failure paths, and session management ensuring predictable behavior and seamless navigation across functional modules.

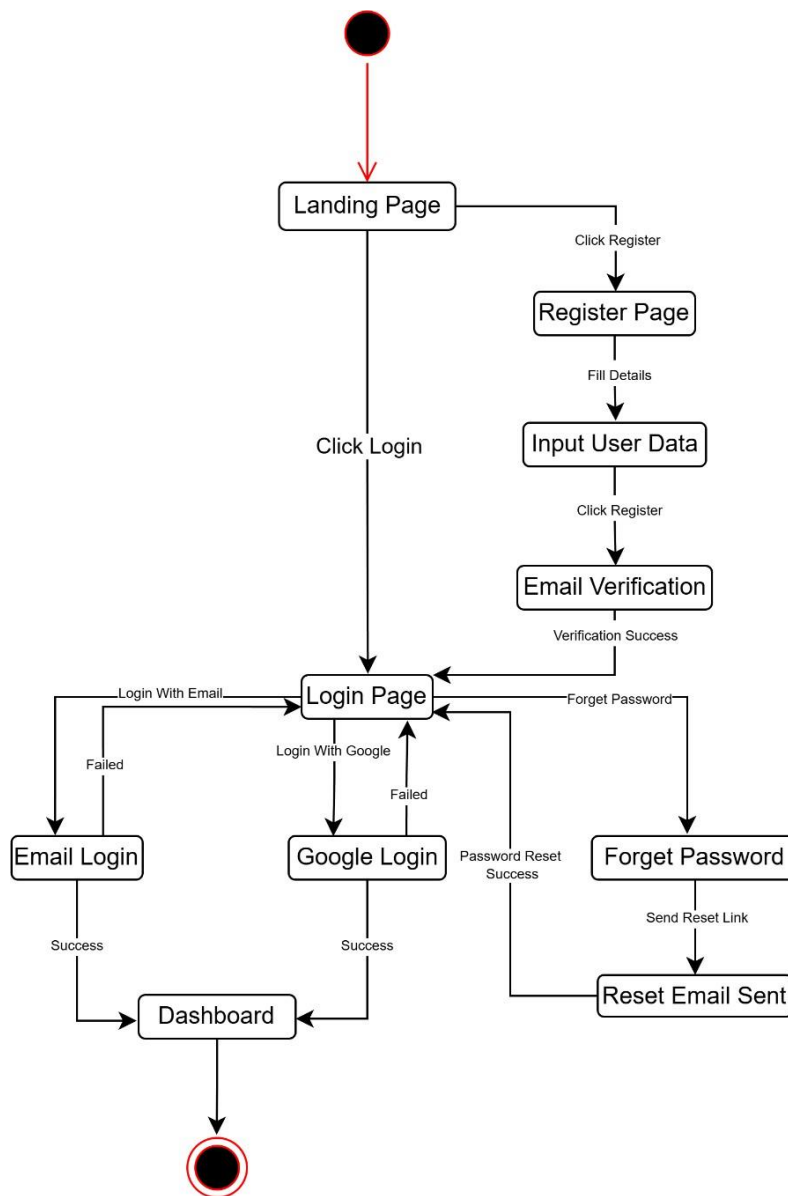


Figure 11: State Transition Diagram 1

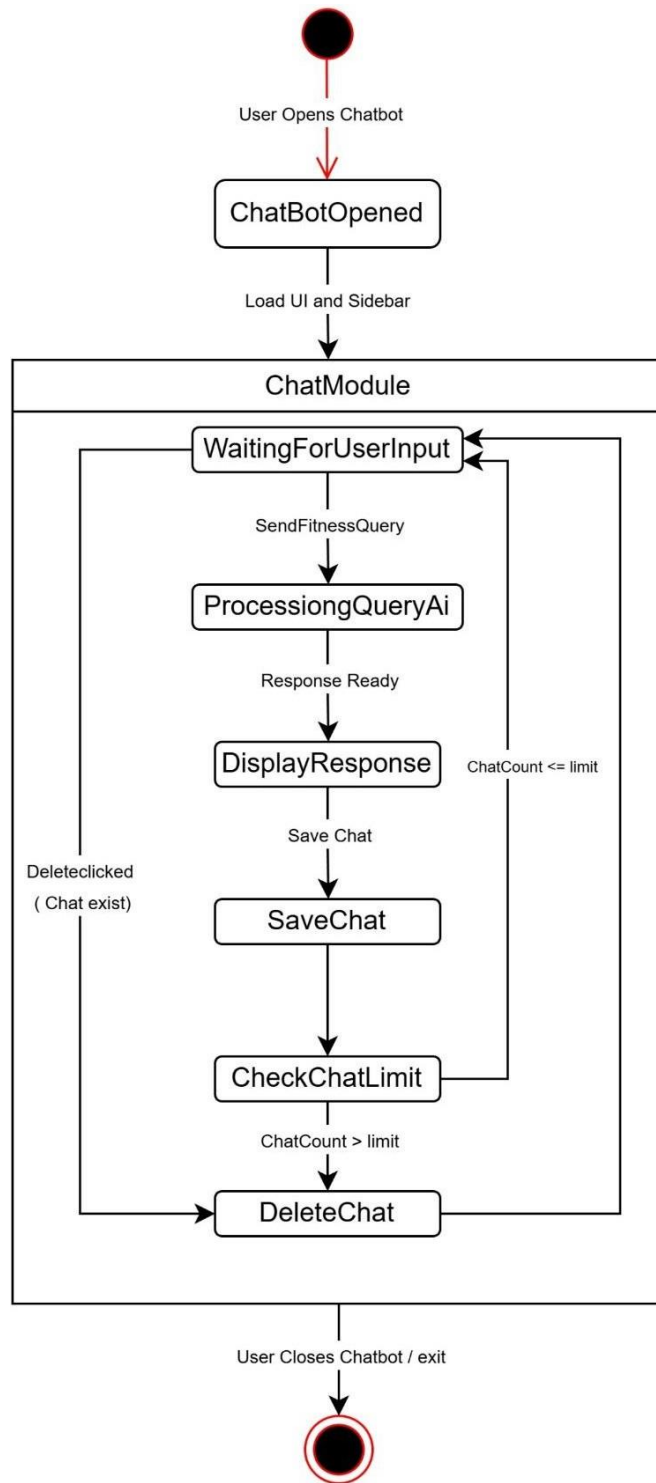


Figure 12: State Transition Diagram 2

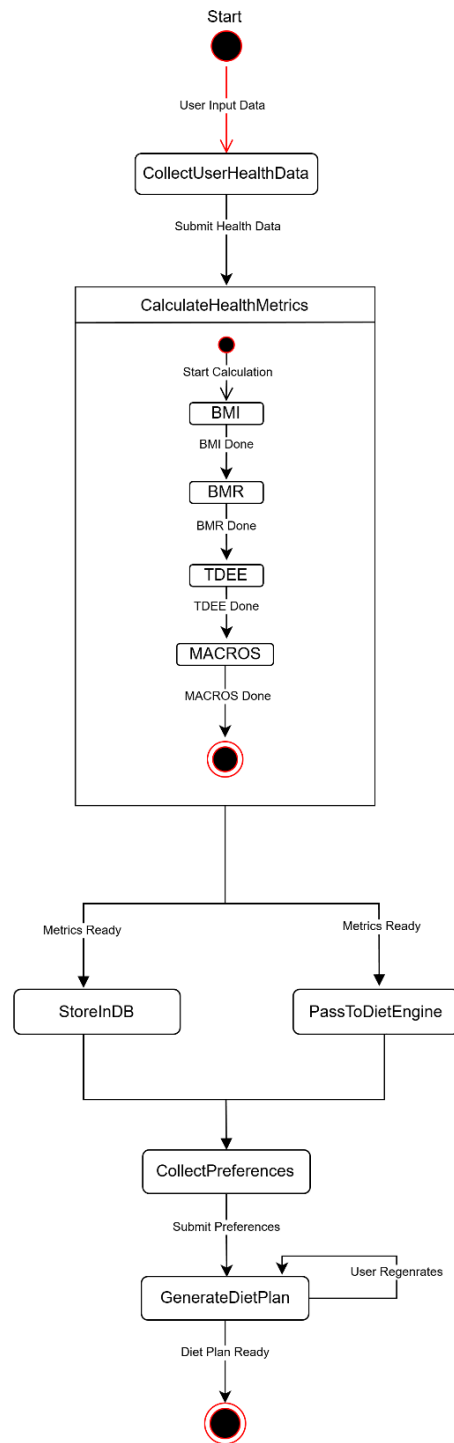


Figure 13: State Transition Diagram 3

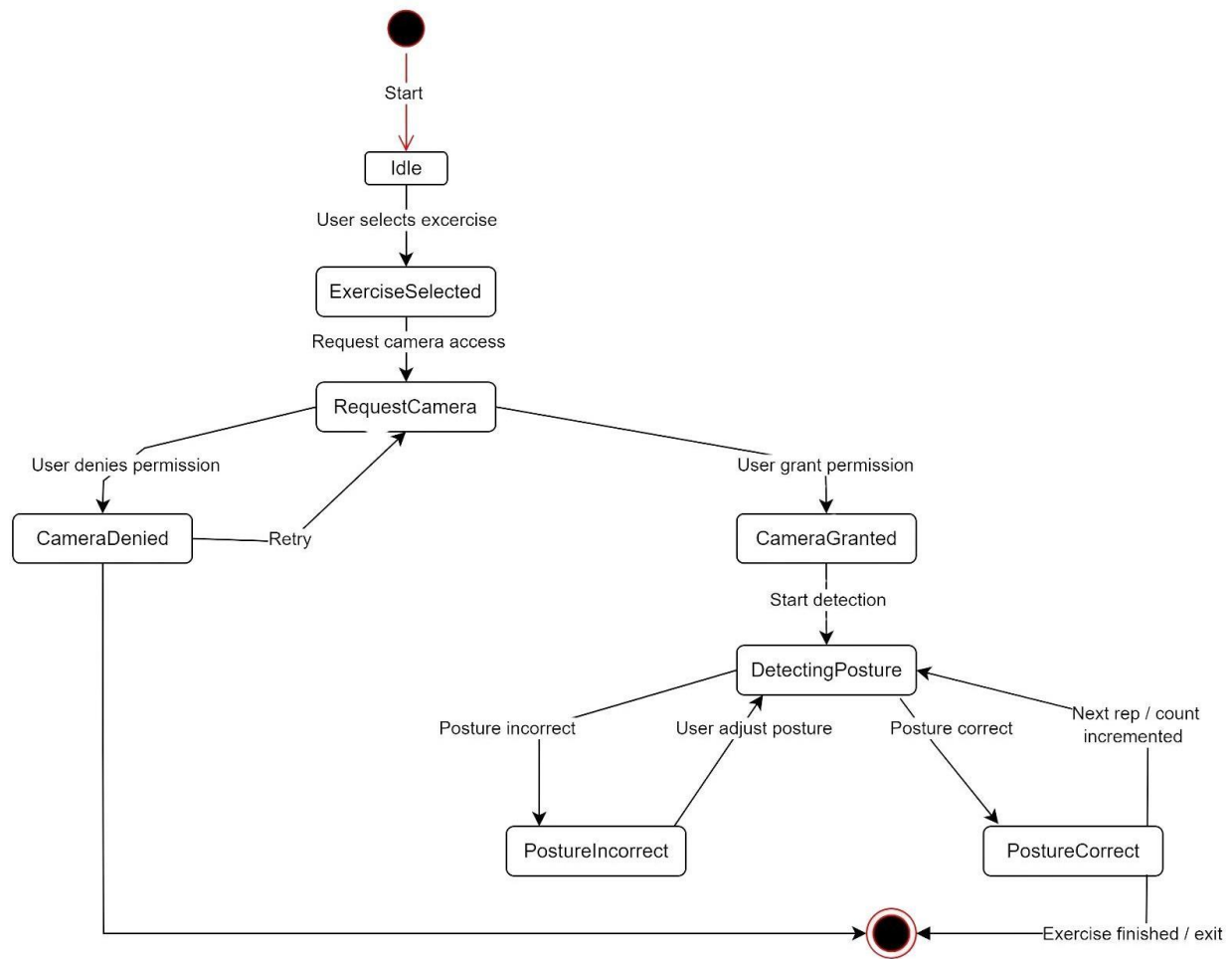


Figure 14: State Transition Diagram 4

3.3 Architectural Design

PoseFit adopts a Three-tier client-server architecture to ensure separation of concerns, maintainability, and scalability. The system is logically divided into presentation, application logic, and data layers, allowing independent development and testing of each component. This structure supports all core modules of the system, including authentication, chatbot interaction, health metrics calculation, diet plan generation, posture detection, feedback handling, and admin management, while following standard MERN stack conventions.

3.3.1 UML Class Relationship Diagram

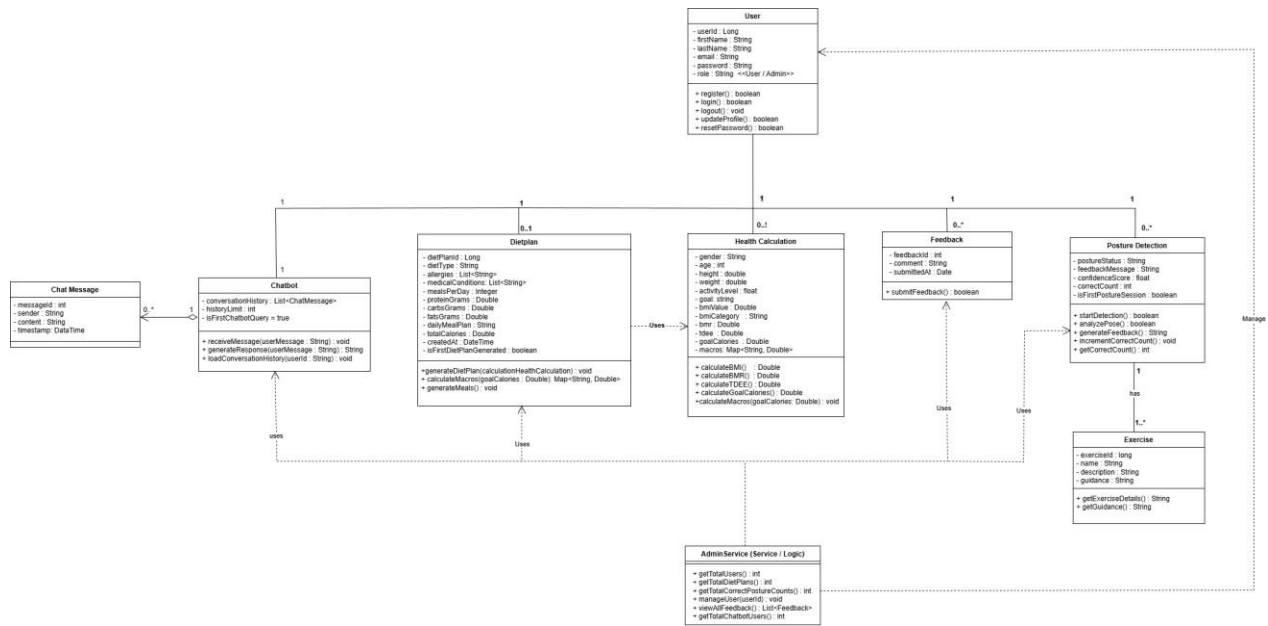


Figure 15: UML Class Relationship Diagram

3.3.2 UML Component Diagram

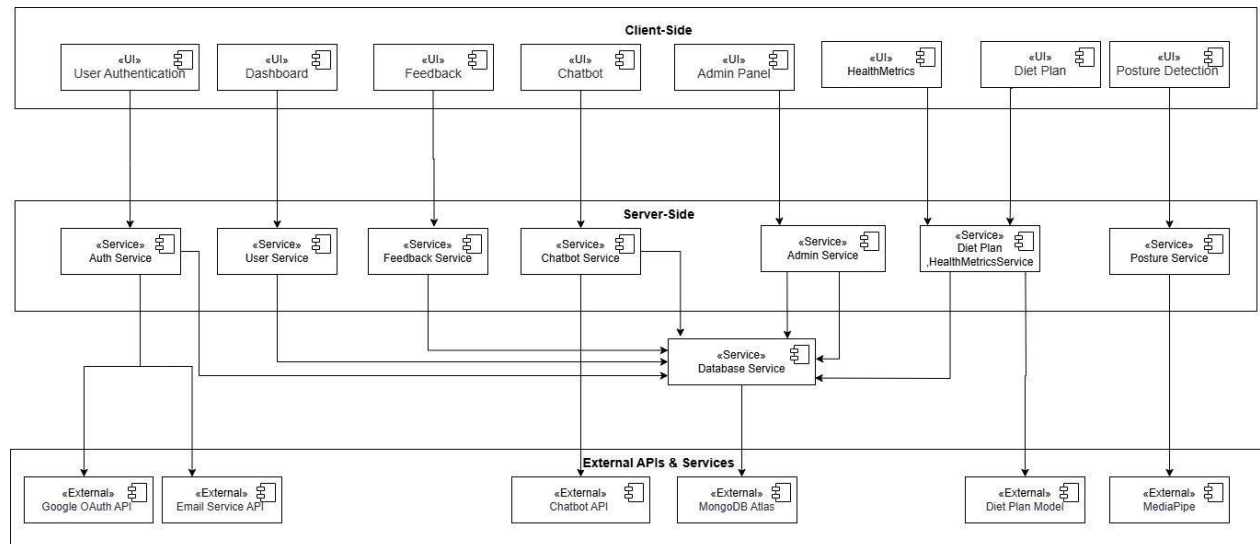


Figure 16: UML Component Diagram

3.3.3 Layered Architecture (3-Tier)

Table 31: Layered Architecture with Responsibilities

Layer	Component / Technologies	Responsibilities
Presentation Layer	React.js, Tailwind CSS	Provides user interfaces for authentication, chatbot interaction, diet planning, posture detection, feedback submission, and admin access.
Application Layer	Node.js, Express.js, LLM (Grok API), Email Service, MediaPipe (TensorFlow.js)	Handles business logic including authentication, health metrics calculation, diet plan generation, chatbot processing, posture analysis, feedback handling, and admin operations.
Data Layer	MongoDB (NoSQL)	Stores user profiles, health metrics, generated diet plans, limited chatbot records, posture usage analytics, and user feedback securely.

3.3.4 MVC

The MVC pattern is used internally within the application layer to organize backend code. The Model includes only stored data entities, while the View represents user interfaces and Controllers manage application logic.

Table 32: MVC Components

MVC Component	Module Assigned
Model	User, Health Metrics, DietPlan, Feedback, ChatMessage.
View	Login Interface, User Dashboard, Chatbot Interface, Health Metrics UI, Diet Plan UI, Posture Detection View, Admin Panel

Controller	Authentication Controller, Chatbot Controller, Diet Controller, Posture Controller, Feedback Controller, Admin Controller
------------	---

3.4 Data Design

PoseFit uses a NoSQL (MongoDB) document-based data design to store and manage system information. The database design represents core real-world entities such as users, health metrics, diet plans, chatbot messages, and feedback in the form of collections. Each collection stores structured documents using unique identifiers (ObjectId), enabling efficient storage, retrieval, and linkage of related records. The data design ensures that user-specific information (health metrics, diet plans, feedback, and chat messages) remains associated with the correct user through `user_id` references. This supports traceability, consistency, and controlled access across the system.

3.4.1 ER Diagram

The Entity Relationship (ER) diagram represents the major stored entities of PoseFit and how they relate to each other. It models the database structure by defining entities, their key attributes, and the cardinalities between them.

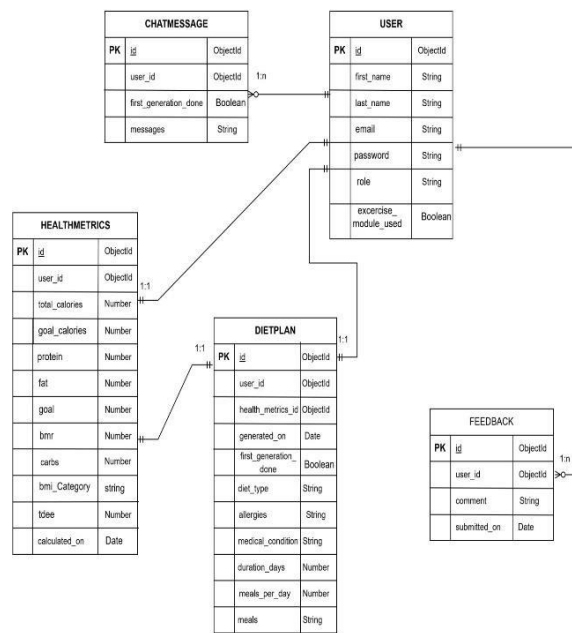


Figure 17: ER Diagram

3.4.2 MongoDB Data Structure

The PoseFit database is organized into the following main collections:

- **User:** Stores user identity and access-related information including name, email, password, role, and exercise_module_used.
- **HealthMetrics:** Stores calculated metrics such as BMI, BMR, TDEE, and Macros.
- **DietPlan:** Stores diet plan details including diet type, allergies, medical condition, duration, meals per day, meals, generation status, and generation date. It links to the related HealthMetrics record using health_metrics_id.
- **ChatMessage:** Stores chatbot message records including user_id, message content, and generation status.
- **Feedback:** Stores user feedback including user_id, comment, and submission date.

3.4.3 Data Processing and Organization

- **Storing Data:** User profiles, health metrics, generated diet plans, feedback comments, and chat messages are stored in MongoDB after validation by the application layer.
- **Retrieving Data:** The system retrieves user-linked records to display dashboard information, diet plan results, chatbot conversation records, and feedback history.
- **Updating Data:** Health metrics and diet plans are updated when users recalculate values or regenerate plans. The exercise_module_used field in the User record is updated to reflect module usage for administrative monitoring.

3.4.4 Data Dictionary

Alphabetical List of System Entities or Major Data with Types and Descriptions:

Table 33: Data Dictionary

Terminology	Description
ChatMessage	Entity for storing chatbot communication data.

ChatMessage.first_generation_done	Boolean: indicates whether the first chatbot response generation has been completed.
ChatMessage.id	ObjectId: unique identifier of the chat message record.
ChatMessage.messages	String: stores chatbot conversation content/messages.
ChatMessage.user_id	ObjectId: reference to the associated User.
DietPlan	Entity for storing diet plan details generated for the user.
DietPlan.allergies	String: stores allergy information provided by the user.
DietPlan.diet_type	String: specifies selected diet type.
DietPlan.duration_days	Number: total duration of diet plan in days.
DietPlan.first_generation_done	Boolean: indicates whether diet plan generation has been completed.
DietPlan.generated_on	Date: stores diet plan generation date.
DietPlan.health_metrics_id	ObjectId: reference to the related HealthMetrics record.
DietPlan.id	ObjectId: unique identifier of the diet plan record.
DietPlan.meals	String: stores generated meals content.
DietPlan.meals_per_day	Number: number of meals per day.
DietPlan.medical_condition	String: stores medical condition information provided by the user.
DietPlan.user_id	ObjectId: reference to the associated User.
Feedback	Entity for storing user feedback regarding the system.

Feedback.comment	String: feedback text submitted by the user.
Feedback.id	ObjectId: unique identifier of the feedback record.
Feedback.submitted_on	Date: feedback submission date.
Feedback.user_id	ObjectId: reference to the associated User.
HealthMetrics	Entity for storing calculated health metrics of the user. Only one record is maintained per user and replaced upon a new save.
HealthMetrics.bmi_Category	String: BMI category label.
HealthMetrics.bmr	Number: Basal Metabolic Rate value.
HealthMetrics.calculated_on	Date: date when metrics were calculated/saved.
HealthMetrics.carbs	Number: calculated carbohydrates value.
HealthMetrics.fat	Number: calculated fats value.
HealthMetrics.goal	Number/String: selected goal value.
HealthMetrics.goal_calories	Number: goal-based calories value.
HealthMetrics.id	ObjectId: unique identifier of the health metrics record.
HealthMetrics.protein	Number: calculated protein value.
HealthMetrics.tdee	Number: Total Daily Energy Expenditure value.
HealthMetrics.total_calories	Number: total calories calculated.
HealthMetrics.user_id	ObjectId: reference to the associated User.

User	Entity representing a registered user of the system.
User.email	String: user email address.
User.exercise_module_used	Boolean: indicates whether the exercise/posture module was used (usage indicator for admin monitoring).
User.first_name	String: user first name.
User.id	ObjectId: unique identifier of the user record.
User.last_name	String: user last name.
User.password	String: user password field (stored securely).
User.role	String: defines role such as User/Admin.

Object-Oriented (OO) Approach: Objects, Attributes, Methods, & Method Parameters:

1- Object: User

Attributes:

- userId : ObjectId
- first_name : String
- last_name : String
- email : String
- password : String
- role : String
- exercise_module_used : Boolean

Methods:

- register()
- login()
- logout()
- updateProfile()

2- Object: HealthMetrics**Attributes:**

- id : ObjectId
- user_id : ObjectId
- total_calories : Number
- goal_calories : Number
- protein : Number
- fat : Number
- carbs : Number
- goal : Number
- bmr : Number
- tdee : Number
- bmi_Category : String
- calculated_on : Date

Methods:

- calculateBMI()
- calculateBMR()
- calculateTDEE()

- calculateMacros()

3- Object: DietPlan

Attributes:

- id : ObjectId
- user_id : ObjectId
- health_metrics_id : ObjectId
- generated_on : Date
- first_generation_done : Boolean
- diet_type : String
- allergies : String
- medical_condition : String
- duration_days : Number
- meals_per_day : Number
- meals : String

Methods:

- generateDietPlan()
- updateDietPlan()

4- Object: ChatMessage

Attributes:

- id : ObjectId
- user_id : ObjectId
- first_generation_done : Boolean

- messages : String

Methods:

- storeMessage()
- fetchMessages()

5- Object: Feedback**Attributes:**

- id : ObjectId
- user_id : ObjectId
- comment : String
- submitted_on : Date

Methods:

- submitFeedback()
- viewFeedback()

3.5 User Interface Design

The user interface design of the PoseFit system is presented from the user's perspective. PoseFit is a web-based application accessible through standard web browsers. After login, users access all system features through a central dashboard with clear navigation options. The dashboard offers clear navigation options for modules such as health metrics, diet planning, posture detection, chatbot interaction, and feedback. The interface is designed to be simple, consistent, and easy to use for all users.

3.5.1 Screen Images

Landing, Register, and Login Page

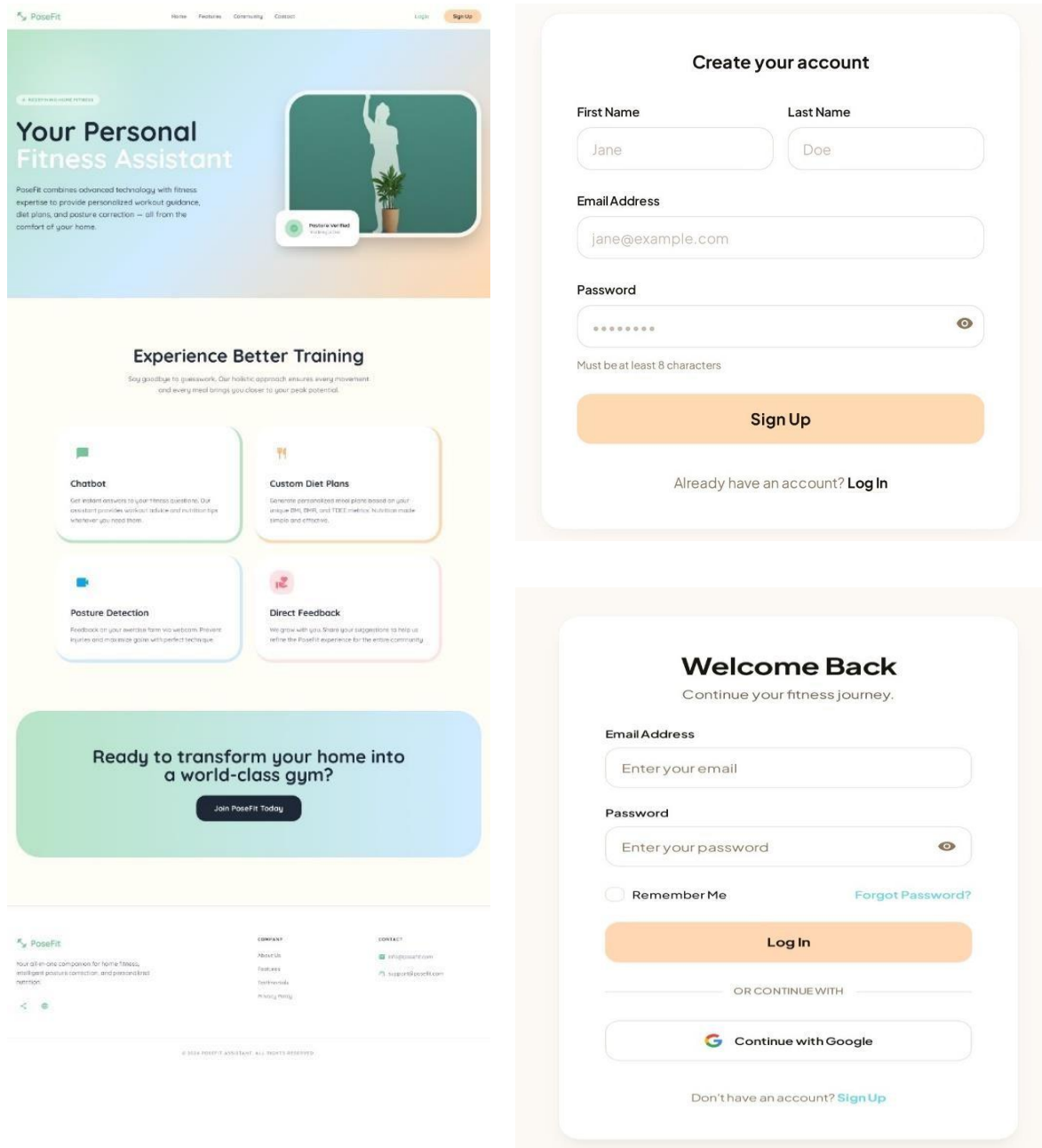


Figure 18: Landing, Register, and Login Page

Dashboard, Diet Plan Page

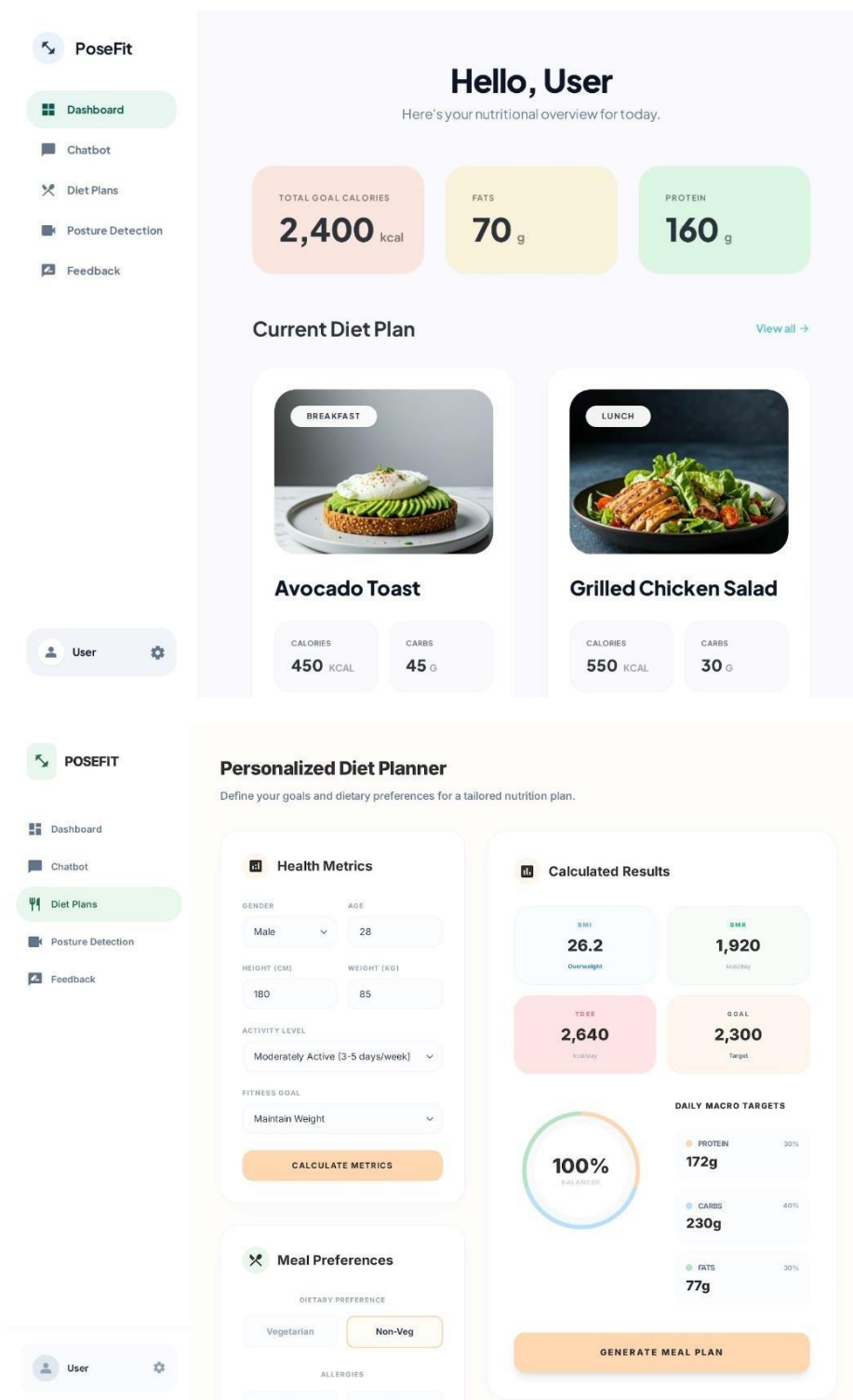


Figure 19: Dashboard and Diet Plan Page

Posture Page

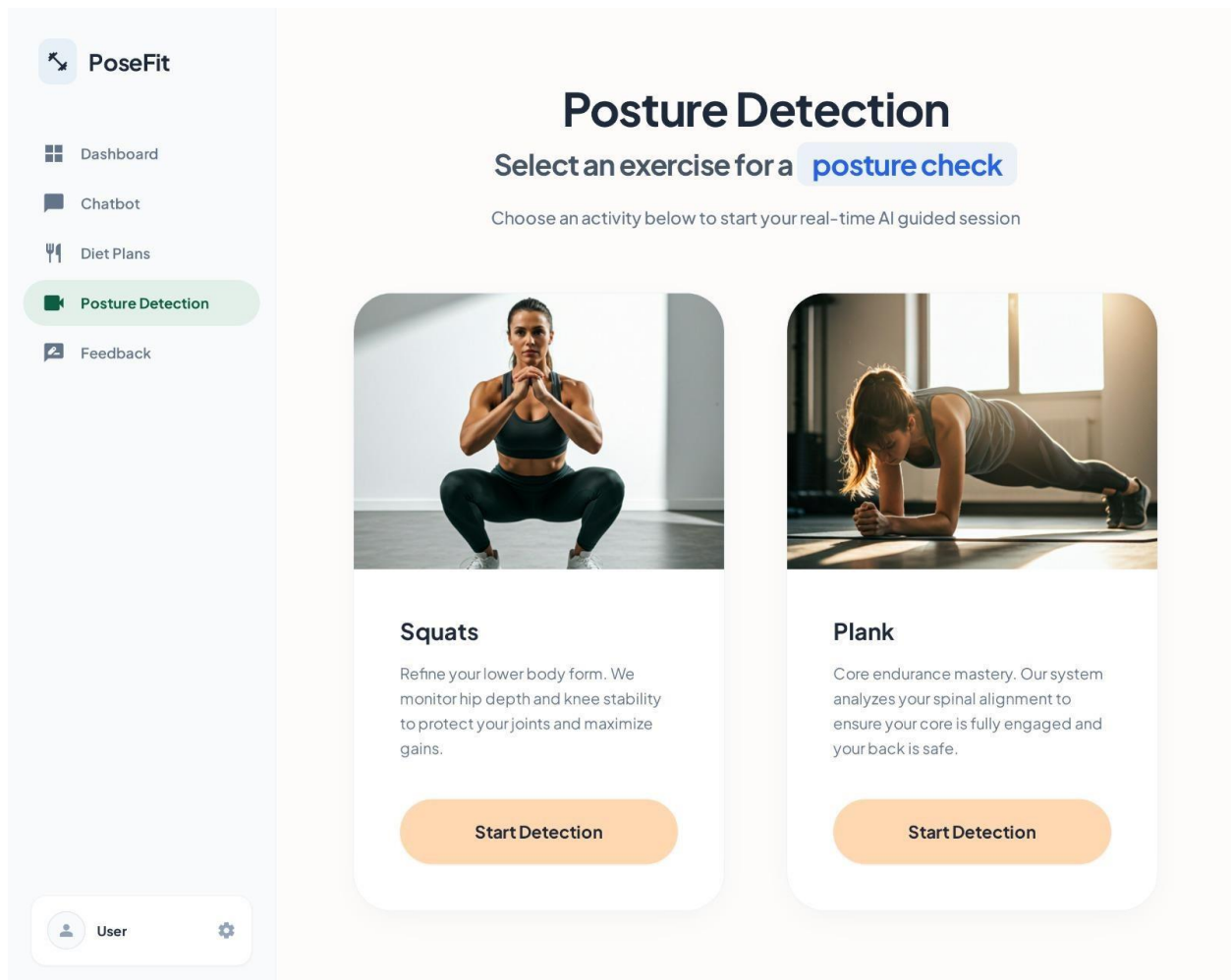


Figure 20: Posture Page

3.5.2 Screen Objects and Actions

Use Case 1: User Login

Screen Objects:

- Email input field.
- Password input field.
- “Remember Me” checkbox.
- “Forget Password” link.

- Login button.
- “Continue with Google” button.
- Sign Up link.

User Actions:

- The user enters a registered email and password.
- The user clicks the Login button to access the system.
- The user may select Continue with Google for Google-based authentication.
- The user may click Forgot Password to reset credentials.
- The system validates input and displays success or error messages.

Use Case 2: Generate Diet Plan

Screen Objects:

- Gender selection dropdown.
- Age input field.
- Height input field.
- Weight input field.
- Activity level dropdown.
- Fitness goal dropdown.
- Calculate Metrics button.
- BMI, BMR, TDEE, and Goal result cards.
- Dietary preference selection (Vegetarian / Non-Vegetarian).
- Allergy selection options.
- Generate Meal Plan button.

User Actions:

- The user enters personal health details such as age, height, weight, gender, and activity level.
- The user selects a fitness goal and clicks Calculate Metrics.
- The user selects dietary preferences and allergies.
- The user clicks Generate Meal Plan to receive a personalized diet plan.

3.6 Behavioral Model

1- Sequence Diagram

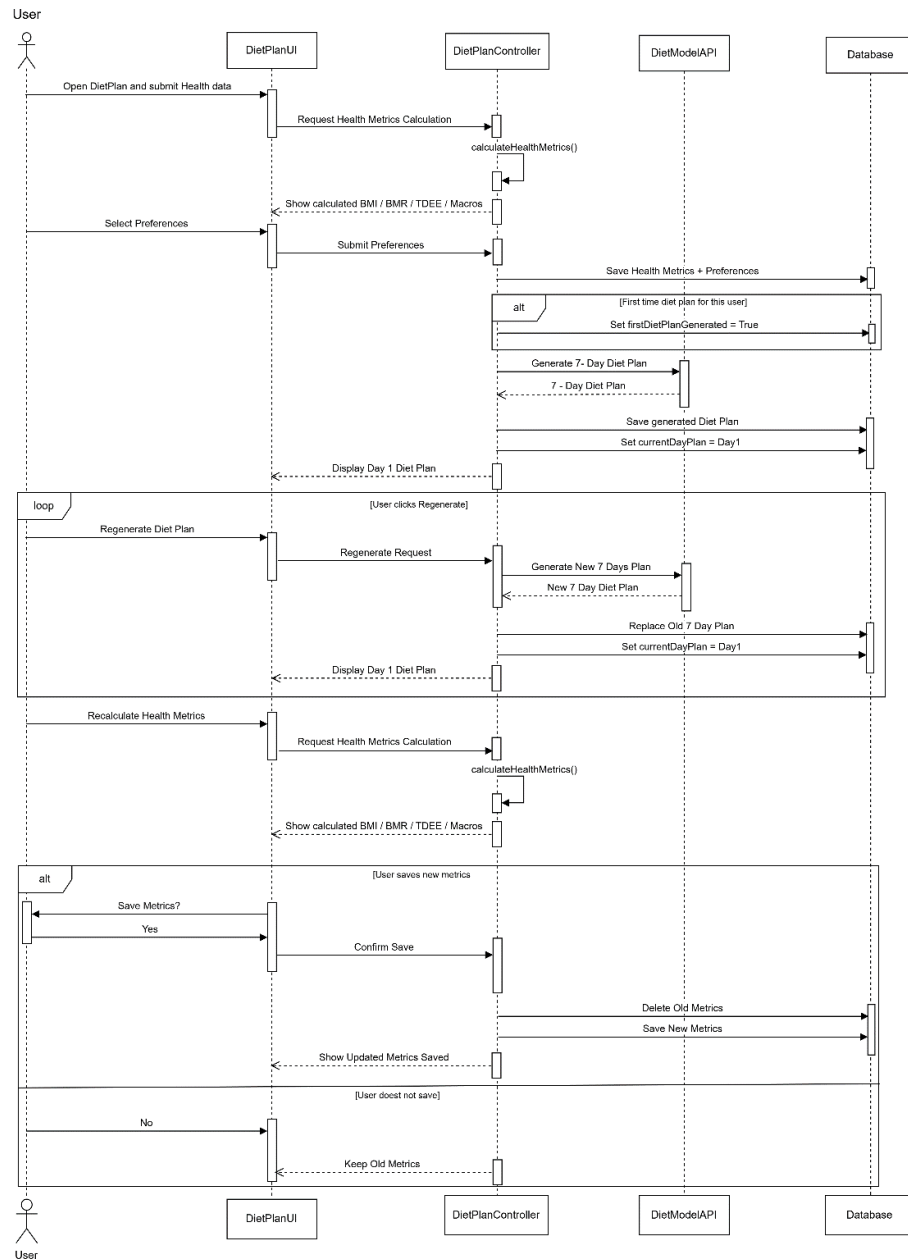


Figure 21: Sequence Model

2- State Transition Diagram

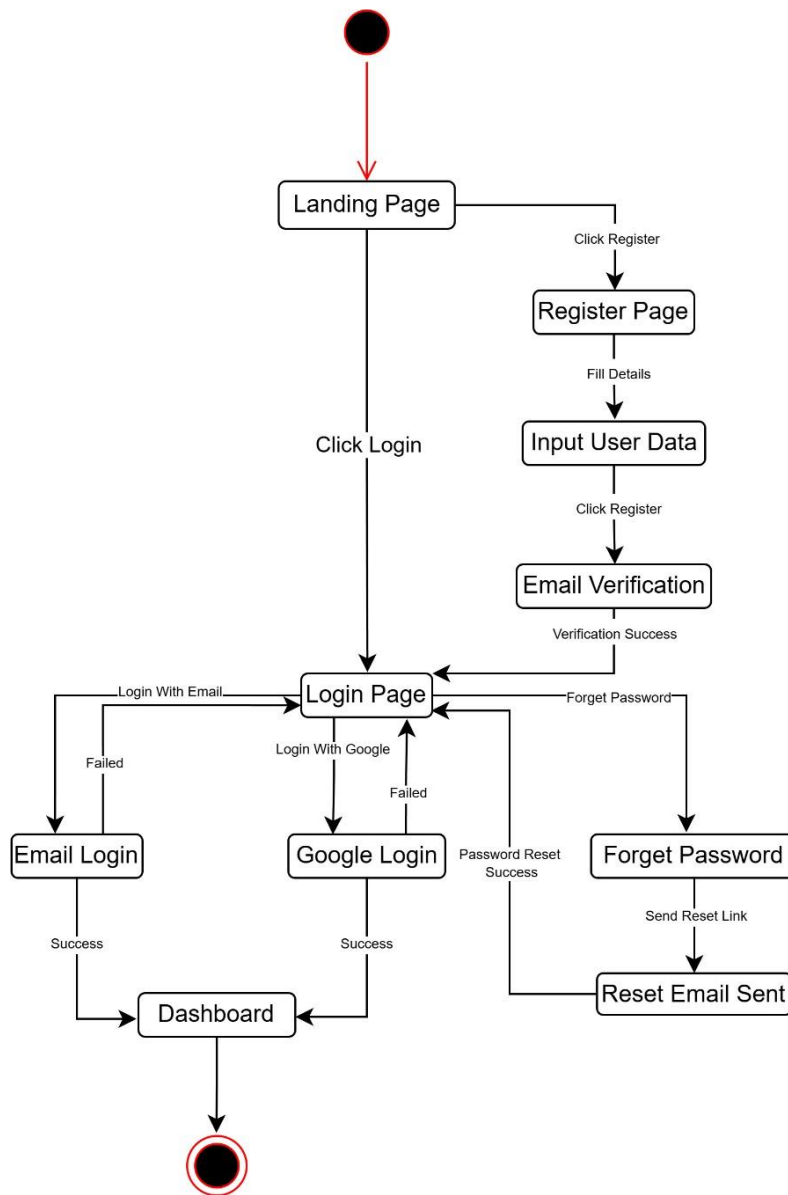


Figure 22: State Transition Model

3.7 Design Decisions

The key design decisions made during the development of the PoseFit system are explained. Each decision was made to ensure simplicity, maintainability, scalability, and alignment with the project

requirements and scope.

Object-Oriented Design Approach

The system follows an object-oriented (OO) design approach. This approach was selected because it allows the system to be modeled using real-world entities such as User, HealthMetrics, DietPlan, ChatMessage, and Feedback. Object-oriented design improves modularity and reusability and makes the system easier to understand, maintain, and extend. The use of classes, attributes, and methods also aligns with the MERN stack implementation and the UML class diagram.

Client-Server Architecture

PoseFit uses a three-tier client-server architecture consisting of presentation, application, and data layers. This design separates user interface logic from business logic and data storage. Such separation improves system organization, simplifies debugging, and allows independent modification of each layer without affecting the overall system.

Technology Stack Selection (MERN Stack)

The MERN stack (MongoDB, Express.js, React.js, Node.js) was selected due to its compatibility with web-based applications and support for full-stack JavaScript development. React.js provides a responsive user interface, while Node.js and Express.js efficiently handle backend logic and API communication. MongoDB supports flexible and scalable data storage suitable for evolving system requirements.

Database Design Choice

A NoSQL document-based database (MongoDB) was chosen instead of a relational database. This decision allows flexible schema design and easy storage of semi-structured data such as health metrics, diet plans, chatbot messages, and feedback. The system stores only the most recent health metrics and diet plan per user, which simplifies data management and aligns with the project scope.

Algorithmic and Processing Decisions

Health metrics such as BMI, BMR, and TDEE are calculated using standard mathematical formulas to ensure accuracy and reliability. Diet plans are generated based on calculated metrics and user inputs. Posture analysis functionality is implemented as a module within the application layer to provide posture analysis and guidance without storing session-level data, keeping the system lightweight and efficient.

Security and Access Control

Authentication and access control are handled within the application layer. Role-based access is implemented to differentiate between normal users and administrators. This design ensures that sensitive operations and administrative features are accessed only by authorized users.

3.8 Summary

This chapter described the design of the PoseFit system. It explained the system architecture, data design, and major design decisions. A three-tier client-server architecture was adopted to separate the user interface, application logic, and data storage. The chapter also presented the data dictionary, defining the main entities and object-oriented components used in the system. Design decisions such as the use of the MERN stack, object-oriented approach, and database structure were justified to support system requirements. This chapter provides the design foundation for the system implementation discussed in the next chapter.

